Defination: OOP as an approach that provides a way of ~~modulating~~ modularizing programs by creating partitioned memory area for both data & functions that can be used as templates for creating copies of such modules on demand.

Thus, an object is considered to be a partitioned area of computer memory that stores data & set of operations that can access that data. Since, the memory partitions are independent the objects can be used in a variety of different programs without modifications.

Concepts of OOP :

a] Object - Objects are the basic runtime entities in an object oriented system. They may represent a person, a place, a bank account, a table of data on any item that the program has to handle, they may also represent user define data like vector, time or list. Object take up space in a memory & have an associated address like a structure in C.

Each object contains data & code to manipulate that data. Objects can interact without having to know details of each others data or code. It is sufficent to know the type of message accepted & type of response return by the object.

b] Classes - Just mentioned that objects contain data & code to manipulate that data. The entire set of data & code of an object can be made user defined datatype with the help of a class. Infact objects are variables of type class, once a class has been defined we can create any no. of objects belonging to that class. Each object is associated with the data of type class with which they are created. A class thus a collection of objects of similar type.

c] Data Abstraction & Encapsulation - The wrapping up of a data & functions into a single unit is known as encapsulation. Data encapsulation is the most striking feature of class. The data is not accessible to the outside world & only

those functions which are wrapped in the class can access it. This insulation of data from direct access by the program is called data hiding.

Abstraction refers to the act of representing essential features without including the background details or explaination. Classes use the concept of abstraction & are defined as list of abstract attributes such as size, weight & cost & functions to operate on this attributes. They encapsulate all the essential properties of the objects that are to be created. The attributes are sometimes called data members because they hold information. The function that operate on this data are called member functions.

Since, the classes use the concept of data abstraction they are known as abstract datatype (ADT).

d] Inheritance - It is the process by which objects of one class acquire properties of objects of another class. It supports the concept of hierarchical classification. The principle behind these sort of division is that each derived class shares common characteristics with the class from which it is derived.

The concept of inheritance provides the idea of reusability, these means that we can add additional features to an existing class without modifying it. These is possible by deriving a new class from the existing one. The new class will have the combined features of both the classes.

e] Polymorphism - This is another important OOP concepts. Polymorphism means the ability to take more than one form. An operation may exhibit different behaviours in different instances. The behaviour depends upon the types of data used in the operation. Eg: Consider the operation of addition for two numbers the operation will generate sum. If the operands are strings then the operation produce third string by concatination. The process of making an operator to

DATE 04 07 2024

exhibit different behaviour in different instances is known as operator overloading.

f] Dynamic Binding: Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding also known as "late binding" means that the code associated with a given procedure call is not known until the time of the call at runtime, it is associated with polymorphism & inheritance. A function call associcted with a polymorphic reference depends on a dynamic type of that reference.

g] Message Passing: An OOP consist of set of objects that communicate with each other. The process of programming in an object oriented language therefore involves following basic steps:

1] Creating classes that define objects & their behaviour.
2] Creating objects from class definations.
3] Establishing communication among objects.

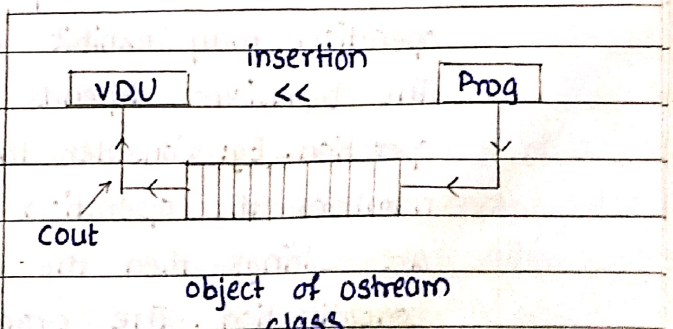
Objects communicate with one another by sending & recieving information, much the same way as people pass messages to one another.

A message for an object is a request for execution of procedure & therefore we invoke a function in the recieving object that generates the desired result. Message passing involves specifing the name of the object, function (message) & the information to be sent.

P1. Program to display "Welcome to C++" message.

→

```
#include <iostream>
#include <stdio>
#include <stdlib>
using namespace std;
void main()
{
    cout << "Welcome to C++";
```



Note: cout << '\n'
cout << 'A'
cout << 4.6
cout << "Program"

DATE 05 07 2024

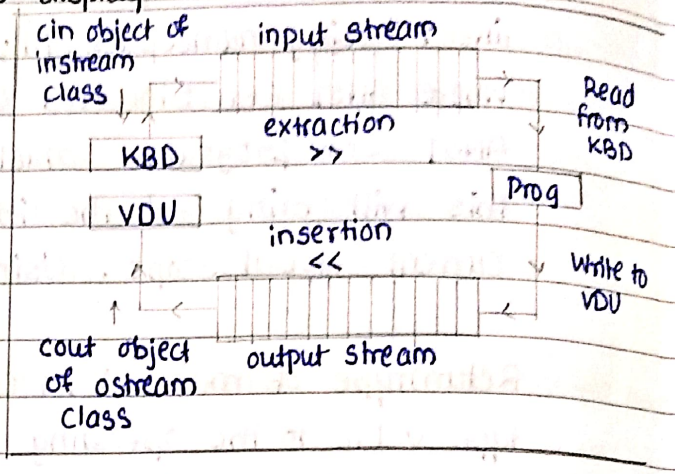
```
    return;
}
```

o/p: Welcome to C++

P2. Write a program to input & display a number.

→

```
#include <iostream>
#include <stdio>
#include <stdlib>
using namespace std;
void main()
{
  cout << "Enter No.:";
  int no;
  cin >> no;
  cout << "\n Number:";
  cout << no;
  return;
}
```



P3. Write a program to find out maximum from two inputed numbers.

→

```
#include <iostream>
#include <stdio>
#include <stdlib>
using namespace std;
void main()
{
  cout << "Enter 2 Nos:";
  int a, b;
  cin >> a >> b; // Cascading of extraction operator.
  cout << "\n Max No.:";
  if (a > b)
    cout << a;
  else
    cout << b;
  return;
}
```

```
cin >> a >> b; (can also be written as)
⇒ cout << "No1:";
  cin >> a;
  cout << "No2:";
  cin >> b;
```

Namespace: Namespace is a new concept introduced by ANSI C++ Standard Committee. This defines a scope for the identifiers that are used in a program. For using the identifiers defined in the namespace scope we must include the using directive like: "using namespace std;". Here, std is the namespace where ANSI C++ Standard Class libraries are defined. All ANSI C++ programs must include this ~~detective~~ directive this will bring all the identifiers defined in std to the current global scope. Using & namespace are the keyword.

Returntype of main(): In C++, main() returns an integer type value to the operating system. Therefore every main() in C++ should end with return 0; statement. Otherwise a warning or an error might occur. Since main() returns an integer type value, returning type for main() is explicity specified as int. NOTE: The default return type for all functions in C++ is int.

P4. Input 10 numbers compute sum & average.

→
```
#include <iostream>
#include <stdio>
#include <stdlib>
using namespace std;
void main()
{
   cout << "\n Enter Numbers:";
   int i=1 , x, tot=0;
   do{
      cin >> x;
      tot+=x;
   }while(++i <= 10)
   cout << "\n Sum:" << sum;
   cout << "\n Avg:" << tot/10;
   return;
}
```

Cascading of I/O Operators: In the above program we have used insertion operator repeatedly in the last statement for printing result. The statement first sends the string "In Sum" to cout & then sends the value of tot. The multiple use of insertion operator in one statement is called cascading. When cascading ~~can~~ an output operator we should ensure necessary blank statement between different items. Using cascading stylesheet technique we can write the statements as follows:

```
cout << " In Total " << tot << "Avg: " << (tot/10);
```

P5. Write a program to find out min. & max. from 10 inputed nos

→

```
#include <iostream>
#include <stdio>
#include <stdlib>
using namespace std;
void main()
{
  cout << "Enter Numbers";
  int i=1 , x, min , max;
  do{
    cin >>x;
    if (i==1)
      min = max = x;
    else
     {
      if (x>max)
        max= x;
      else if (x<min)
        min= x;
     } i++;
  }while(i<=10);
  cout << "In Minimum:" << min;
  cout << "In Maximum:" << max;
  return;
}
```

DATE

P6. Write a program to find no. of days in inputed month no.

→
```
#include <iostream>
# include <stdio>
# include <stdlib>
void main()
{
  int mno, days;
  cout << "Enter Month Number:";
  cin >> mno;
  if (mno < 1 || mno > 12)
     cout << "\nInvalid Input";
  else
  {
     switch (mno)
     {
        case 2:
            days = 28; break;
        case 4:
        case 6:
        case 9:
        case 11:
            days = 30; break;
        default:
            days = 31;
     }
  }
  cout << "\n Days" << days << " in month " << mno;
  return;
}
```

Dynamic Initialization of Variables: In C a variable must be intialized using a constant expression & the C compiler would fix the initialization code at the time of compilation.

For Example:

PAGE

DATE 09 07 2024

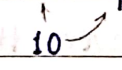
```
#include <iostream>
#include <stdio>
#include <stdlib>
void main()
{
   cout << "\n Enter City Name: ";
   char a[50];
   cin >> a;
   int n = strlen(a);
   cout << "\n City:" << a << "\n Length of String:" << n;
}
```

C++ permits intialization of variables at runtime. This is reffered to as dynamic initialization. In C++ a variable can be initialized at runtime using expressions at the place of declaration. In the above program it is shown by a highlight Dynamic initialization is extensively used in OOP.

Reference Variables:- C++ introduces a new kind of variable known as the refrence variable. A reference variable provide Alias (alternative name) for a previously defined variable reference variable is created as follows:

<data_type> & <ref_variable> = <variable>;

Eg: int x = 10;
int & y = x;



'x' is a int type variable that has already been declared, 'y' is the alternative name declared to represent the variable 'x'. Both the variables refer to the same object in memory.

A reference variable must be initialized at the time of declaration. This establishes the correspondance between the reference & data object which it names. It is important to note that the initialization of variable is completely different from signed to it.

C++ assign an additional meaning to the symbol (&). Here '&' is not address operator, the notation 'int &' means

reference to int. The measure application in reference variable is in passing arguments to functions.

Ex:
```
void main()
{
    int x=10;
    int & y=x;                          o/p: x:7
    cout << "\n x:" <<x;
    x+=5;
    cout << "\n x:" <<y;
    y|=2;
    cout << "\n x:" <<x;
}
```

Scope Resolution Operator (::):- Like C, C++ is also a block structured language. Blocks & scopes can be used in constructing programs. We know that the same variable name can be used to have different meanings in different blocks. The scope of the variable extends from the point of its declaration till the end of the block containing the declaration. A variable declared inside a block is said to be local to that block.

In C, the global version of variable cannot be accessed from inner block. C++ resolves this problem by introducing a new operator (::) called the scope resolution operator. This can be used to uncover the hidden variable. It takes the following form: :: <variable_name>.

This operator allows access to the global version of a variable. A measure application of the scope resolution operator is in the classes to identify the class to which a member function belongs.

Ex:
```
int x=1000;
void main()                             o/p:
{
    int x=10;
    {
        int x=100;
```

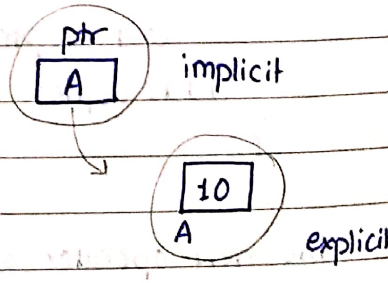
```
        cout << "In x:" << x << "\t" << ::x;
    }
    cout << "In x:" << x << "\t" << ::x;
    return;
}
```

Memory Management Operators: C uses 'malloc' & 'calloc' functions to allocate memory dynamically at runtime. Similarly, it uses the function 'free' to free dynamically allocated memory. We use dynamic allocation technique when it is not known in advance how much of memory is need. C++ support this function, it also defines to unary operators **new** & **delete** that perform the task of allocating & freeing the memory in the better & easier way. Since this operators manipulate memory on the free store they are also known as **free store operators.**

A object can be created by using new & destroyed by using delete as & when required. A data object created inside a block with new will remain in existance until it is explicity destroyed by using delete. Thus the life time of an object is directly under our control & is unrelated to the block structure of program. The new operator can be used to create objects of any type.

i] <ptr_variable> = new <data-type>

Ex:
```
void main()
{
    int * ptr;
    ptr = new int;
    cout << "\n Enter No.: ";
    cin >> *ptr;
    cout << "\n No:" << *ptr;
    *ptr = *ptr + 5;
    cout << "\n No:" << *ptr;
    delete ptr;
}
```



DATE

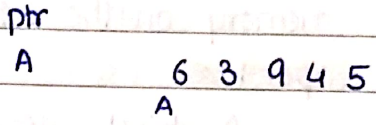
ii] <ptr_variable> = new <data-type>(<initial_value>);

Ex: void main()
{
int *ptr;
ptr = new int(10);
cout << "In No:" << *ptr;
*ptr = *ptr+5;
cout << "In No:" << *ptr;
delete ptr;
}

iii] Use of new operator to make available memory in continuous manner.

<ptr_variable> = new <data-type> [<no_of_elements>];

Ex: void main()
{
int *ptr;
ptr = new int[5];
int i=0;
while (i<5)
{
cout<< "In Data";
cin >> ptr[i];
i++;
}
cout <<"In Data:";
for(i=0; i<5; i++)
cout << ptr[i] << " ";
delete []ptr;
}



The new operator offers following advantages over the malloc():

1] It automatically computes the size of the data object we need not ~~used~~ use operator "sizeof".

2] It automatically returns the correct pointer type, so that there

is no need to use a type cast.

3] It is possible to initialize the object while creating the memory space.

4] Like any other operators "new" & "delte "delete" can be overloaded.

Manipulators: (iomainip.h)

Manipulators are operators that are used to format the data display, following are most commonly used manipulators:

setw(n): sets column width for insertion

setprecision(n): sets decimal places from fixed width.

setfill(ch): sets filling character to fill blanks from fixed width.

endl(): Inserts newline character.

P7→

```
void main()
{
  clrscr();
  cout << 1234 << " " << "kirankumar" << " " << 68.92 << "\n";
  cout << 6 << " " << " Deepa" << " 78.93 << "\n";
  return;
}.
```

```
1 2 3 4  k i r a n k u m a r  6 8 . 9 2 0 0 0 0
6  D e e p a  7 8 . 9 3 0 0 0 0
```

P8→

```
void main()
{
  clrscr();
  cout << setw(4) << 1234 << " << setw(10) << setiosflags(ios::left) <<
    "kirankumar" << " " << resetiosflags(ios::left) << setw(6) <<
     setprecision(2) << setfill('#') << 9.38 << setfill(' ') << endl;
  cout << setw(4) << 6 << " " << setw(10) << setiosflags(ios::left) << "Deepa
    << resetiosflags(ios::left) << setw(6) << setprecision(2) << 8.63 <<
     endl;
}
```

DATE 11/07/2024

1 2 3 4	kirankumar	##9.88
6	Deepa	8.63

Functions in C++ :→ function play an important role in C program development. Dividing a program into functions is one of the major principles of top down structured programming. Another advantage of using function is that it is possible to reduce the size of program by calling & using at different places in the program.

When the function is called control is transfered to the first statement in the function body. The other statements in the function body are the executed & the control returns to the main program when the closing brace is encountered. C++ is no exception, infact C++ has add many new features to functions to make them more reliable & flexible. C++ function can be overloaded to make it perform different task depending on the arguments passed to it.

The main(): C doesn't specify any return type for the main() which is the starting point for the execution of the program. In C++ the main() returns, a value of type int to the operating system. C++ therefore explicitly defines main() with returning type int. Ex:

```
int main()
{
    ⋮
  return 0;
}
```

The functions that have return value should use the return statement for termination. Since the returning type of function is int by default the keyword int in the main() header is optional. Most C++ compiler will generate an error or warning if their is no return statement.

Many OS tools/test the return value to determine if their is any problem. The normal convention in that & exit value of 0 is the program ran succeful, while a non-zero value means

PAGE

classmate

their is a problem.

Function Prototyping: Function prototyping describes the function interface to compiler by giving details such as the no. of & type of arguments & type of return value. With function prototyping a template is always used when declaring & defining function. When a function is called the compiler uses templete to assume that proper argument is passed & the return value is treated correctly. An violation in matching the arguments or the result type will be caught by the compiler at type of compilation itself.

Call by Reference: In traditional C a function call passes argument by value. The called function create a new of variable & copies the values of argument in them. The function doesn't have access to the actual variables in the calling program & can only work of copies of values. These mechanism is fine if the function doesn't need to alter the values of original variables in the calling program. But there may arise situation where we would like to change the values of variable in the calling program.

Provision of the reference variable in C++ permits as to pass parameters to the function by reference. When we pass arguments by reference the formal argument in the called function become "aliases" to the actual arguments in the called function. This means that when the function is working with its own arguments it is actually working on original data.

Pg1 Program to demonstrate function calling by passing value.

→

```
void swap (int a , int b)
{
    int c = a;
    a = b;
    b = c;
}
```

```
void main()
{
   cout << "Enter 2 No's:";
   int p,q;
   cin >> p >> q;
   cout << "\n Before function call: \n P:" << p << "\t Q:" << q;
   swap(p,q);
   cout << "\n After function call: \n P:" << p << "\t Q:" << q;
   return;
}
```

P10] Function call by reference.

→

```
void swap(int &a, int &b)
{
   int c = a;
      a = b;
      b = c;
}
void main()
{
   cout << "Enter 2 No.s:";
   int p,q;
   cin >> p >> q;
   cout << "\n Before function call: \n P:" << p << "\t Q:" << q;
   swap(p,q);
   cout << "\n After function call: \n P:" << p << "\t Q:" << q;
   return;
}
```

Inline Function: One of the objectives of using function in a program is to save some memory space which becomes appreciable when a function is likely to be called many times. However, everytime a function is called it takes a lot of extra time in exiquting a series of instructions for task such as jumping to the

functions saving registers, pushing argument into the stack & returning to the calling function. When a function is small, a sustainial percentage of execution time maybe spent in such overheads.

C++ has solution to this problem, to eliminate the cost of calls to small functions. C++ proposes a new feature called inline function. An inlinie function is a function that is expanded inline when it is invoked .i.e. the compiler replaces the function call with corresponding function code. The inline functions are defined as follows:

```
inline <function_header>
{
    ____________;
    ____________;
    ____________;
}
```

It is easy to make function inline all we need to do is to prefix the keyword inline to the function definition. All inline functions must be defined before they are called.

Note that the inline keyword sends the request not the command to a compiler. The compiler may ignore this request if the function definition is too long or too complicated & compile the function as a normal function.

Following are some of the situations where inline expansion may not work,

1) For functions returning value if a loop, a switch or go to exist.
2) For functions not returning values, if a return statement exist.
3) If function contain static variables.
4) If inline functions are recursive.

Ex:
```
inline void display()
    {
      cout << "\nWelcome";
      return;
    }
inline int sqr(int n)
    {
```

```
    return(n*n);
}
inline int cube(int n)
{
    return (n*n*n);
}
void main()
{
    display();
    cout << "In Square:" << sqr(7);
    cout << "In Cube:" << cube(7);
    return;
}
```

Function with Default Arguments:- C++ allows us to call a function without specifying all its arguments in such cases a function assigns default value to the parameter which doesn't have matching argument in the function call. Default values are specified when the compiler looks at the prototype to see how many arguments the function uses & alerts the program for possible default values. The default value is specified in a manner synthetically similar to a variable initialization.

A default argument is checked for type at the time of declaration & evaluated at the time of call. One important point to note is that only the trailing arguments can have default values & therefore we must add defaults from right to left. We cannot provide default value to a particular argument in a middle of an argument list.

Default arguments are useful in situations where some arguments always have the same value.

Advantages:

1) We can use default arguments to add new parameters to the existing functions.

2] Default arguments can be used to combine similar functions into one.

```
Ex: void repli(char ch, int n)    // n=5 (by default).
    {
       cout <<"\n";
       for (int i=1; i<=n; i++)
           cout << ch;
    }
    void main()
    {
       repli('#');
       repli('>',3);
       repli('+',7);
    }
```

Output:
```
# # # # #
> > >
+ + + + + + +
```

Function Overloading:- Overloading refers to the use of the same thing for different purposes. C++ also permits overloading of functions. This means that we can use the same function name to create functions that perform a variety of different tasks. This is known as function polymorphism in OOP.

Using the concept of function overloading we can design a family of functions with one function name but with different argument list. The function ~~eat~~ who perform different operations depending on argument list in the function call. The correct function to be invoked is determined by checking the number & type of argument but not on the function type.

A function call first matches the prototype having the same number & type of arguments & then calls the appropriate function for execution. A best match must be unique. The function selection involves following steps:

1] The compiler first tries to find an exact match in which the types of actual arguments are the same & use that

functions.

2] If an exact match is not found the compiler uses the integral promotions to have the actual arguments such as:
- char to int
- float to double
- To find a match.

3] When either of them fails the compiler tries to uses the built in conversions to the actual arguments & then uses the function whose match is unique. If the conversion is possible to have multiple matches then the compiler will generate an error message.

Ex: void display (long);
void display (double);
and a fn call,
display (10);

will cause an error because 'int' argument can be converted to either long or double. Thereby, creating an ambigious situation as to which version of display should be used.

4] If all the steps fails, the compiler will try the user define conversions in combination with integral promotions & built in conversions to find unique.

```
Ex: void display()
    {
      cout << "In Welcome";
    }
    void display (int a)
    {
      cout << "In Data:" << a;
    }
    void display (double d)
    {
      cout << "In Data:" << a;
    }
    void display (long int a)
```

```
{
  cout << "In Data: " << a;
}
void main()
{
    display();
    display(x);
    display(10);
    display(9.2);
    return;
}
```

- Classes & Objects :- One of the unique feature of the C language is structures. They provide a method for packing together data of different types. A structure is a convinent tool for handling a group of logically related data items. It is a user define datatype with a template that serves to define its data properties. Once the structure type has been defined we can create variables of that type using declarations that are similar to the built-in type declaration.

Ex:
```
struct student
{
  int rno;
  char nm[15];
  double mrk;
};
```

The keyword struct declares 'student' as a new datatype that can hold three fields of different datatypes. This fields are known as structure members or elements. The identifier 'student' which is refer to as structure name can be used to create variables of type student. Structures can have arrays, pointers or structures as members.

Limitation of C structure:

1] The standard C doesn't allow the struct datatype to be

treated like built in types.

2] We can't use operators over two structure type variables.

3] Another important limitation of C structure is that they don't permit data hiding. Structure members can be directly accessed by the structure variables by any function anywhere in there scope, in other words structure members are public members.

Extension to Structure: C++ supports all the features of structures as defined in C but C++ has expanded it capabilities further to suit its OOP philosophy. It attempts to bring the user define types as close as possible to the built-in datatypes & also provides facility to hide data which is one of the main principles of OOP. Inheritance a mechanism by which one type can inherit characteristics from other types is also supported by C++.

In C++ a structure can have both variables & functions as members. It can also declare some of its members as private. So that they can't be accessed directly by the external functions.

In C++ the structure names are stand alone & can be used like any other type names. In other words the keyword struct can be omited in declaration of structure variable.

Ex: student a;

Remember this is error in C.

C++ incorporates all this extensions in another user defined type known as class. There is very little syntatical difference between structure & classes in C++. Since class is a specially introduced datatype in C++, most of the C++ programmers tend to use the structure for holding only data & classes to hold both data & functions.

Specifying a Class: A class is a way to bind the data & its associated functions together. It allows the data to be

hidden from external use. When defining a class we are creating a new abstract datatype that can be treated like any other built-in datatype. Generally, class specification has two parts:

1] Class declaration.

2] Class function definition.

The class declaration describes the type & scope of its members. The class function definition describes how the class functions are implement, following is general form of class declaration.

```
Ex: class <class_name>
    {
      private:
        <data_members>;
        <member_function>;
      public:
        <data_members>;
        <member_function>;
    };
```

~~They~~ The keyword class specific that what data is an abstract datatype. "<class_name>" the body of class is enclosed within braces & terminates by semicolon (;). The class body contains declaration of variables & functions. These are collectively called class members. They are usually grouped under two sections name 'private' & 'public' to denote which are public.

'Private' members are accessed only from within the class where as 'public' members are accessed from outside the class. The use of the word private is optional. By default the members of class are private. The variables declared inside the class known as data members & the functions are known as member functions. Only the members functions can have access to private data members & private functions however public members can be accessed from outside

DATE 18/07/2024

the class.

P11] Program to implement class student.

→

```
#include <iostream>
#include <stdlib.h>
using namespace std;
class student
{
private:
    int no;
    char nm[15];
public:
    void setData()
    {
        cout << "\n Roll No:";
        cin >> no;
        cout << "\n Name:";
        cin >> nm;
    }
    void display()
    {
        cout << "\n Roll No:" << no;
        cout << "\n Name:" << nm;
    }
};
void main()
{
    student a;
    a.setData();
    a.display();
}
```

keyword

type name

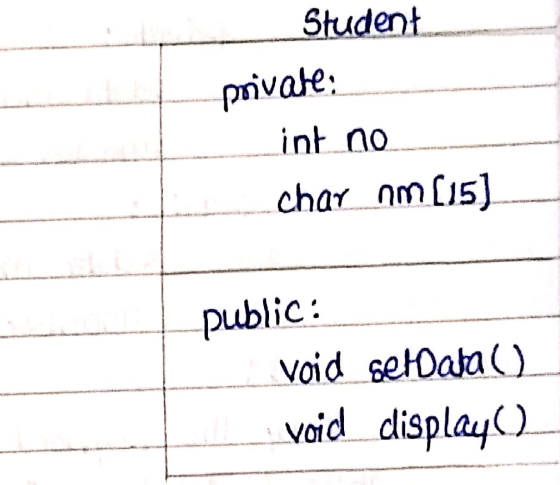
begin of class definition

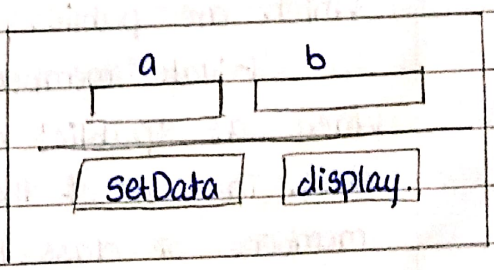
visibility label

data members

member function

end of class definition





DATE

P12] Program to implement class book.

→

```
#include <iostream>
#include <stdlib.h>
using namespace std;
class book
{
  private:
     int bno;
     char bnm[15];
     char banm[15];
     int bprice;
  public: void setData(){
     cout<<"\n Book No.:";
     cin>>bno;
     cout<<"\n Book Name:";
     cin>>bnm;
     cout<<"\n Author Name:";
     cin>>banm;
     cout<<"\n Book Price:";
     cin>>bprice;
     }
    void display()
    {
       cout<<"\n Book No:"<<bno;
       cout<<"\n Book Name:"<<bnm;
       cout<<"\n Author Name:"<<banm;
       cout<<"\n Book Price:"<<bprice;
    }
};
void main()
{
   book a;
   a.setData();
   a.display();
}
```

classmate

PAGE

DATE 19 07 2014

P13] Member function definition outside the class.

→

```
#include <iostream>
# include <stdlib.h>
using namespace std;
class Book
{
   int bno;
   char bnm[15];
   int bprice;
   public:
      void setData();
      void setData(int);
      void display;
      int getNo()
      {
         return bno;
      }
      int getPrice()
      {
         return bprice;
      }
      char *getName()
      {
         return bnm;
      }
};
void Book :: setData(int no)
{
   bno = no;
   cout << "\n Enter Book Name:";
   cin >> bnm;
   cout << "\n Enter Book Price:";
   cin >> bprice;
}
```

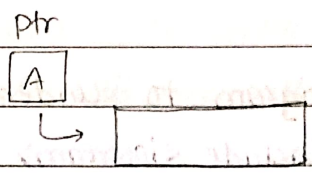
PAGE

classmate

DATE

```
void Book :: setData()
{
    cout << "In Book Data In Enter Book No:";
    cin >> bno;
    cout << "In Enter Book Name:";
    cin >> bnm;
    cout << "In Enter Book Price:";
    cin >> bprice;
}
void Book::display()
{
   cout <<"In Book Data In No:"<< bno <<"In Name:" <<bnm <<
    "In Price "<<bprice;
}
/*void main()
 {
    Book b;
    b.setData();
    b.display();
 }
void main()
{
    Book *ptr;
    ptr = new Book();
    ptr -> setData();
    ptr -> display();
    delete ptr;
}
// array of objects of type book
void main()
{
   Book b[5]; int i=0;
   while (i<5)
   {
```

comment out main as only one main gets compiled at once



```
        b[i].setData(i+1);
        i++;
    }
    for(i=0 ; i<5 ; i++)
        b[i].display();
}
// Use of new operator to make available continous memory
   for array of object of type book.
void main()
{
    Book *b= newBook[5]; int i=0;
    while(i<5)
    {
        b[i].setData(i+1);
        i++;
    }
    for(i=0; i<5; i++)
        b[i].display();
    delete[]b;
    return;
}
```

P14] Program to understand 'this' pointer.

→

```
#include <iostream>
#include <stdlib.h>
using namespace std;
class Number
{
  int x,y;
  void swap()
  {
     int z=x;
        x=y;
        y=z;
  }
```

DATE

```
public:
  void setData()
  {
    cout << "In x:";
    cin >> x;
    cout << "In y:";
    cin >> y;
  }
  void setData(int p)
  {
    x = p; y = p;
  }
  void setData(int x, int y)
  {
    this -> x = x;
    this -> y = y;
  }.
```

** // C++ uses a unique keyword called 'this' to represent an object that invokes the member function. 'this' is a pointer that points to an object for which this function was called. This unique pointer is automatically passed to member function when it is called. The pointer 'this' acts as an implicit argument to call the member functions.

```
  void setData (Number &t)
  {
    x = t.x;
    y = t.y;
  }
  void display ()
  {
    cout << "In x:" << x;
    cout << "In y:" << y;
  }
```

classmate

```
int getX() {return x;}
int getY() {return y;}
void setx (int a) {x=a;}
void sety (int a) {y=a;}.
Void callswap()
{
   swap();
}
};

void main()
{
   Number p,q,r;
   p.setData();
   p.display();
   q.setData();
   q.display();
   r.setx(100);
   r.sety(100);
   r.display();
}
```

P15] Program with two classes.

→

```
class Point
{
  int x,y;
  void swap()
  {
    int z = x;
      x = y;
      y = z;
  }
  public:
    void setData()
```

```
{
    cout<<"\nx:";
    cin>>x;
    cout<<"\ny:";
    cin>>y;
}
void setData(int P)
{
    x=p; y=p;
}
void setData(int x, int y)
{
    this->x = x;
    this->y = y;
}
void setData( Point &t)
{
    x = t.x;
    y = t.y;
}
void display()
{
    cout<<"\nx:"<<x;
    cout<<"\ny:"<<y;
}
int getx() {return x;}
int gety() {return y;}
void setx(int a) {x=a;}
void sety (int a) {y=a;}
Void callswap()
{
    Swap();
}
};
```

DATE 24 07 2024

```
class Rectangle
{
    Point p;
    Point q;
  public:
    void setData()
    {
        cout << "In Point-p: ";
        p.setData();
        cout << "In Point q:";
        q.setData();
    }
    void setData (Point &a, Point &b).
    {
        p=a; q=b;
    }
    void setData (int x1, int y1, int x2, int y2).
    {
        p.setData (x1, y1);
        q.setData (x2, y2);
    }.
    int getWidth()
    {
        return (q.getx() - p.getx());
    }
    int getHeight()
    {
        return (q.gety() - p.gety());
    }
    int area()
    {
        return getWidth() * getHeight();
    }
};
```

```
void main ()
{
  Rectangle a;
  a.setData (10, 10, 120, 100);
  cout <<" In Width:" << a.getWidth ();
  cout <<" In Height:" << a.getHeight ();
  cout << " In Area:" << a.area ();
}
```

P16]

→

```
#include <iostream>
#include <stdlib.h>
using namespace std;
class Result
{
   int rno;
   int mrk[5];
  public:
    void init ();
    void setData (int);
    void setData ();
    void setMarks ();
    void display ();
    int total ();
    int avg ();
    int getNo () {return rno;}
}; int equals (int no) { return (rno == no); }
    int isMore (Result &t)  { return  total () > (t.total ()); }
};
void main ()
{
   Result d, p;
   d.setData (101);
   d.display ();
```

:: → Scope Resolution Operator.

DATE

```
    p.getData();
    p.display();
    if (d.equals(101))
      cout << "\n Found";
    else
      cout << "\n Not Found";
    if (d.isMore(p))
      cout << "\n d is at top";
    else
      cout << "\n p is at top";
}

void Result :: init()
{
    rno = 0;
    for (int i=0; i<5 ; i++)
      mrk[i] = 0;
}
int Result :: total()
{
    int i=1, tot = mrk[0];
    while (i<5)
    {
      tot += mrk[i];
      i++;
    }
    return tot;
}
int Result :: avg()
{
    return total()/5;
}
void Result :: display()
{
```

PAGE

DATE

```
    cout << "\n Roll No: " << rno;
    cout << "\n Marks \n";
    for (int i=0; i<5; i++)
        cout << "  " << mrk[i];
    cout << "\n Total:" << total();
    cout << "\n Avg: " << avg();
}
void Result :: setMarks()
{
    cout << "\n Marks \n";
    for ( int i=0 ; i<5 ; i++)
        cin >> mrk[i];
}
void Result :: setData(int n)
{
    rno = n;
    setMarks();
}
void Result :: setData()
{
    cout << "\n Roll No: ";
    cin >> rno;
    setMarks();
}.
```

P17] Class to implement Matrix Operation.

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class Matrix
{
    int x[3][3];
public:
```

```
    void init();
    void setData();
    void display();
    void transpose();
    void add (matrix &, matrix &);
    void sub (matrix &, matrix &);
    void mul (matrix &, matrix &);
};

void Matrix :: init()
{
  int i,j;
  for(i=0; i<3; i++)
  {
     for(j=0; j<3; j++)
        x[i][j]=0;
  }
}
void Matrix :: SetData()
{
  int i,j;
  for(i=0; i<3; i++)
  {
     for(j=0; j<3; j++)
     {
        cout<<"\n x[" <<i <<"][" <<j <<"]:";
        cin >> x[i][j];
     }
  }
}.
void Matrix :: display()
{
  int i,j;
  for(i=0; i<3; i++)
```

```
    }
      for (j=0; j<3 ; j++)
        cout << " " << x[i][j];
      cout << "\n";
    }
}
void Matrix :: transpose()
{
  int i,j,tmp;
  for (i=0; i<3; i++)
  {
    for (j=0; j<i ; j++)
    {
        tmp = x[i][j];
        x[i][j] = x[j][i];
        x[j][i] = tmp;
    }
  }
}
void Matrix :: add (Matrix &p , Matrix &q)
{
  int i, j;
  for (i=0; i<3 ; i++)
  {
    for (j=0 ; j<3 ; j++)
    {
        x[i][j] = p.x[i][j] + q.x[i][j] ;
    }
  }
}
void Matrix :: sub (Matrix &p , Matrix &q).
{
  int i,j;
  for (i= 0 ; i<3 ; i++)
```

```
    {
        for (j=0; j<3; j++)
            x[i][j] = p.x[i][j] - q.x[i][j];
    }
}
void Matrix :: mul (Matrix &p , Matrix &q)
{
    int i,j,k;
    for (i=0; i<3; i++)
    {
        for ( j=0; j<3; j++)
        {
            x[i][j] =0;
            for (k=0; k<3; k++)
                x[i][j]+= (p.x[i][k] * q.x[k][j]);
        }
    }
}

void main()
{
    Matrix a,b,c;
    cout << "\n Data for Matrix a \n";
    a.setData();
    cout << "\n Data for Matrix b \n";
    b.setData();
    c.add(a,b);
    a.display();
    b.display();
    c.display();
    return;
}
```

DATE 27 07 2024

Friend Function: We have been emphasizing throughout this chapter that the private members cannot be accessed from outside the class i.e. a non-member function cannot have an access to the private data of a class. However there could be a situation where we would like two classes to share a particular function. In such situation C++ allows the common function to be made friendly with both the class, thereby allowing the function to have access to the private data of these classes. Such a function need not be the member of any of these classes.

To make an outside function friendly to a class we have to simply declare the function as friend of the class as shown below:

```
class ABC
{ ____;

public:
____;

friend <fn_header>(<arguments>);
};  // declaration.
```

The function declaration of the be preceded to the ~~program~~ keyword friend. The function is defined elsewhere in the program like a normal fn. The function definition doesn't use either the keyword friend / sro(::). The function that are declared with keyword friend are called frend function. The function can be declared as friend in any no. of classes. A friend functio although not a member function has full access rights as the private member of the class.

Characteristics:

1] It is not in the scope of the class to which it has been declared as friend.

2] Since it is not in scope of class it cannot be called using as the object of that class.

3] It can be invoked like a normal function without help of any object.

4] Unlike member functions it cannot access member names directly & has to use an object name & .membership operator. with each member name.

5] It can be declared either in the public or private operator in the class without affecting its meaning.

6] Usually it has object as its argument.

P18→
```
#include <iostream>
#include <stdlib.h>
using namespace std;
class Number
{
    int no;
  public:
    void setData()
    {
        cout << "\nNo:";
        cin >> no;
    }
    void display()
    {
        cout << "\nNo:" << no;
    }
    friend void update(Number &, int);
};
void update(Number &t, int val)
{
    t.no = t.no + val;
}
void main()
{
    Number a;
```

```
    a.setData();
    a.display();
    update(a, 5);
    a.display();
}
```

P19] Regular function friend for both function.

→

```
#include <iostream>
#include <stdlib.h>
using namespace std;
class B; //forward declaration.
class A
{
    int a;
  public:
    void set()
    {
        cout<<"\n A a:";
        cin >> a;
    }
    void display()
    {
        cout <<"\n A a" << a;
    }
    friend void swap (A&, B&);
};
class B
{
   int b;
  public:
    void set()
    {
        cout<<"\n B b:";
        cin >> b;
    }
```

```
    void display ()
    {
      cout <<"\n B b:" <<b;
    }
    friend void swap (A&, B&);
};
void swap(A &p, B &q)
{
    int tmp = p.a;
      p.a = q.b;
      q.b = temp;
}

void main ()
{
    A obja ; B objb;
    obja.set();
    objb.set();
    obja.display();
    objb.display();
    swap (obja, objb);
    obja.display();
    objb.display();
    return;
}
```

P20] Member function of class A is friend to the class B.

→

```
#include <stdio.h>
# include<stdlib.h>
# include <iostream>
using namespace std;
class B;
class A
{
```

DATE 28/07/2024

```
    int a;
  public:
    void set()
    {
       cout <<"\nA a:";
       cin >>a;
    }
    Void display()
    {
       cout <<"\nA a:" <<a;
    }
    void swap(B &);
};
class B
{
    int b;
  public:
    void Set()
    {
       cout <<"\nBb:";
       cin >> b;
    }
    void display()
    {
       cout<<"\nBb:" <<b;
    }
    friend void A :: swap(B &);
};
void A :: swap(B &q)
{
    int r = a;
     q = q.b;
    q.b = r;
}
```

```
void main()
{
    A p; B q;
    p.set(); q.set();
    p.display(); q.display();
    p.swap(q);
    p.display(); q.display();
}
```

P21] Member function of class B is friend to class A.

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class A;
class B
{
  int b;
 public:
  void set()
  {
     cout <<"\nBb:";
     cin >> b;
  }
  void display()
  {
    cout <<"\nB b:" << b;
  }
  void swap(A &)
};
class A
{
   int a;
  public:
```

```
    void set()
    {
        cout <<"\n Aa: ";
        cin >> a;
    }
    void display()
    {
        cout <<"\nAa:" <<a;
    }
    friend void B:: swap(A &);
};
void B :: swap (A &q)
{
     int r = q.a;
      q.a = b;
       b = r;
}

void main():
{
   A p; B q;
   p.set(); q.set();
   p.display(); q.display();
   q.swap(p);
    p.display(); q.display();
}.
```

DATE

- Constructors & Destructors:

1] These are special purpose function.
2] Always to be defined in public section of class.
3] Don't have returning type, not even void.
4] Class name is the function name.
5] Constructors functions are used to initialize objects of the class.
6] Constructor functions can be overloaded.
7] Constructor functions are getting called automatically at the time of object creation.
8] Constructor functions can be called explicitly also.
9] Constructor functions are used to convert basic type data to the user defined datatype.
10] There are three types of constructor functions:
   a] Default Constructor Function: Constructor function with no arguments.
   b] Paramatriced Constructor Fn: Constructor function with atleast one argument.
   c] Copy Constructor Function: Constructor function with same class object as orgument.
11] Destructor functions are used to release external resource associated with an object
12] Destructor Function cannot be overloaded.
13] Destructor Function are getting called automatically when program execution is running out of scope in which objects are created.
14] Destructor function should be prefixed by the sign (~) tiddle.

22] Program to implement class number define appropriate constructors & Destructors.

→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
```

DATE 29/07/2024

```
using namespace std;
class Number
{
    int no;
  public:
    Number()
    {
       no=0;
       cout<<"In Default";
    }
    Number(int a)
    {
       no=a;
       cout<<" Para";
    }
    Number (Number &t)
    {
       Nr no=t.no;
        cout<<"InCopy";
    }
    ~Number()
    {
        cout<<"InDestructor";
    }
    void setData(int n)
    {
       no=n;
    }
    void setData()
    {
       cout<<"In No:";   cin>>no;
    }
    void display()
    {
```

```
    cout << "\n No: " << no;
  }
};

void main()
{
  Number a;     // default
  {
    Number b(10);  // para.
    b.display();
  }
  a.setData();
  a.display();
  Number c(a);  // copy.
  c.display();
}
```

P23] Constructor member function definition outside the class.

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>   ↵   # define N 5
using namespace std;
class Array
{
  int x[N];
 public:
  Array();
  Array(int);
  Array(Array &);
  ~Array() { }
  void setData();
  void display();
  int total();
  int avg()
```

```
    {
        return total()/n;
    }
};

Array::Array()   // default
{
    int i=0;
    while (i<N)
    {
        x[i]=0;
        i++;
    }
}
Array :: Array(int val)   // para
{
    int i=0;
    while (i<N)
    {
        x[i]=0;
        i++;
    }
}
Array :: Array (Array &t)   // copy
{
    int i=0;
    while (i<N)
    {
        x[i]=0;
        i++;
    }
}
void Array :: setData()
{
```

```
void Array :: setData()
{
cout<<"Enter Data";
int i;
while (i<N)
{
cin>>x[i];
i++;
}
```

DATE

```
int i=0;
cout<<"\nData\n";
while (i<N)
{
    cin >> x[i];
    i++;
}
}
void Array :: display()
{
   cout <<"\nData\n";
   for (int i=0; i<N; i++)
       cout<<" " << x[i];
}
int Array :: total()
{
   int i= 1, tot= x[0];
   while (i<N)
     tot += x[i++];
   return tot;
}

void main()
{
   Array p;
   p.setData();
   p.display();
   cout <<"\nTotal:";
   cout <<p.total();
   cout <<"\nAvg:";
   cout << p.avg();
   Array q(p);
   q.display();
}
```

PAGE

DATE 30 07 2024

P24]

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class Matrix
{
    int x[3][3];
  public:
    Matrix ();
    Matrix (Matrix &);
    ~Matrix ( ) { }
     void setData();
      void display();
     Matrix  add (Matrix &);
     Matrix  sub (Matrix &);
     Matrix  mul (Matrix &);
};
void main()
{
   Matrix a,b,c;
   cout <<"\n Data for Matrix a\n";
    a.setData();
   cout <<"\n Data for Matrix b\n";
    b.setData();
    c = a.add(b);
    a. display;
    b. display;
    c. display;
    return;
}
Matrix :: Matrix()
{
  int i,j;
```

```
    for (i=0 ; i<3 ; i++)
    {
        for (j=0 ; j<3 ; j++)
            x[i][j]=0;
    }
}
Matrix :: Matrix (Matrix &t)
{
    int i,j ;
    for (i=0 ; i<3 ; i++)
    {
        for (j=0 ; j<3 ; j++)
            x[i][j] = t.x[i][j];
    }
}
void Matrix :: setData()
{
    int i , j ;
    cout << "\n Matrix Data \n";
    for (i=0; i<3 ; i++)
    {
        for (j=0 ; j<3 ; j++)
            cin >> x[i][j];
    }
}
void Matrix :: display ()
{
    cout << "\nMatrix\n";
    int i,j ;
    for (i=0; i<3; i++)
    {
        for (j=0 ; j<3 ; j++)
            cout <<"  " << x[i][j];
        cout << "\n";
    }
}
```

DATE

```
Matrix Matrix :: add (Matrix &t)
{
   int i,j;
   Matrix q;  // default constructor.
   for (i=0; i<3; i++)
   {
      for(j=0; j<3; j++)
         q.x[i][j] = x[i][j] + t.x[i][j];
   }
   return q;
}
Matrix Matrix :: sub(Matrix &t)
{
   int i,j;
   Matrix q;
   for(i=0; i<3; i++)
   {
      for(j=0; j<3; j++)
         q.x[i][j] = x[i][j] - t.x[i][j];
   }
   return q;
}
Matrix Matrix :: mul(Matrix &t)
{
   int i,j,k;
   Matrix q;
   for (i=0; i<3; i++)
   {
      for (j=0; j<3; j++)
      {
         q.x[i][j]=0;
         for (k=0; k<3; k++)
            q.x[i][j] += (x[i][k] * t.x[k][j]);
      }
   }
   return q;
}
```

classmate

PAGE

DATE 31/07/2024

Q5]

```
#include <stdio.h>
#include <stdlib.h>   #include <string.h>
#include <iostream>
using namespace std;
class Student
{
    int no;
    char nm[15];
    double mrk;
  public:
    Student()
    {
       no=0;
      nm[0]='\0';
       mrk = 0.0;
    }
    Student (int a, char *b, double c)
    {
        no=a;
        strcpy(nm,b);
        mrk=c;
    }
    Student (Student &t)
    {
        no=t.no;
        strcpy(nm, t.nm);
        mrk = t.mrk;
    }
    ~Student() {}
    int getNo () { return no; }
    char * getNm () { return nm; }
    double getMarks() { return mrk; }
    void display()
```

```
    {
        cout <<"\nRoll No:" <<no <<"\n Name:" <<nm <<"\nMarks:" <<mrk;
    }
    void setData ()
    {
        cout <<"\n Roll No:";
        cin>> no;
        setData (no);
    }
    void set Data (int a)
    {
        no = a;
        cout <<"\n Name:"; ~~cin~~ cin>> nm;
        cout <<"\n Marks:"; cin>> mrk;
    }
};

Void main ()
{
    Student a;
    a. setData ();
    a. display ();
    Student b ( 13, "Gaurav", 9.2);
    b. display ();
}
```

H] Create a header file "mytime.h"

→ ~~class~~ # include <stdio.h>

```
# include <stdlib.h>
# include <string.h>
# include <iostream>
using namespace std;
class MyTime
{
```

DATE

```
int h,m,s;
public:
MyTime()
{
    h=m=s=0;
}
MyTime (int a, int b=0, int c=0)
{
    h=a; m=b; s=c;
}
MyTime (long int d)
{
    h= d/3600;
    d= d%3600;
    m= d/60;
    s= d%60;
}
MyTime (const MyTime &t)
{
    h=t.h; m=t.m; s=t.s;
}
~MyTime () { }
int getHr () {return h;}
int getMn() {return m;}
int getSs() {return s;}
longint toSeconds()
{
    return h*3600l + m*60 + s;
}
void setTime (long int a)
{
    h = a/3600;
    a = a%3600;
    m = a/60;
```

PAGE

classmate

```
    s = a%60;
}
void setTime(int a, int b=0, int c=0)
{
    h=a; m=b; s=c;
}
void setTime(const MyTime &k)
{
    h=k.h; m=k.m; s=k.s;
}
void setTime()
{
    cout<<"\nHours:";cin>>h;
    cout<<"\nMinutes:";  cin>>m;
    cout<<"\nSeconds:";  cin>>s;
}
MyTime add(const MyTime &k)
{
    long int a = toSeconds();
    long int b =k.toSeconds();
    MyTime c(a+b);
    return c;
}
MyTime sub (MyTime &t)
{
    long int a= toSeconds();
    long int b = t.toSeconds();
    long int c = labs(a-b);  //long int absolute.
    MyTime d(c);
    return d;
}
Void display()
{
    cout<<"\nHr:"<<h<<"\tMn:"<<m<<"\tSec:"<<s;
}
```

```
int isMore (MyTime &t)
{
    long int a = toSeconds();
    long int b = t.toSeconds();
    return (a>b);
}
int isLess (MyTime &t)
{
    long int a = toSeconds();
    long int b = t.toSeconds();
    return (a<b);
}
};
```

P26) Use "myTime1.cpp"

→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include "MyTime.h"
void main()
{
   MyTime a(7,20,30);
   a.dislay();
   a.setTime();
   a.display();
}
// Use "myTime2.cpp"
void main()
{
   MyTime a,b,c;
   a.setTime();
   b.setTime();
   c=a.sub(b);
   a.display();
```

```
    b.display();
    c.display();
}
```

H] Dynamic Object.

→
```
#include <stdio.h>
#include <iostream>
#include <stdlib.h>
using namespace std;
class dynobj
{
   int noe;
   int *ptr;
 public:
    dynobj()
    {
       noe=0;
       ptr=NULL;
    }
    ~dynobj()
    {
       if (ptr!=NULL)
          delete[] ptr;
    }
    void setData();
    void display();
    int total();
    int avg() {return total()/noe;}
};
void dynobj::setData()
{
   cout<<"Information Count:";
   cin>>noe;
   ptr = new int[noe];
```

DATE

```
    cout << "\n Data: \n";
    for (int i = 0; i < noe; i++)
        cin >> ptr[i];
}
int dynobj :: total()
{
    int i=0, tot=0;
    while (i<noe)
    {
        tot += ptr[i];
        i++;
    }
    return tot;
}
void dynobj :: display()
{
    cout << "\n Data\n";
    for (int i = 0; i<noe; i++)
    c   cout << "" " << ptr[i];
    cout << "\n Total" << total();
    cout << "\n Average" << avg();
}

void main()
{
    dynobj k;
    k.setData();
    k.display();
}.
```

DATE

Operator Overloading & type Conversions:

operator overloading is one of the many exciting features of C++ language. It is an important technique that has enhanced the power of extensibility of C++. We have stated more than once that C++ tries to make the user defined datatypes behave in much the same way as the built-in types. For instance C++ permits us to add two variables of user defined types with the same syntax that is applied to the basic types. These means that C++ has the ability to provide the operators with a special meaning for a datatype. The mechanism of giving such special meanings to an operator is known as operator overloading.

Operator Overloading provides a flexible option for the creation of new definitions for most of the C++ operators. We can overload all the C++ operators except the following:

1] Class Member Access Operators (., .*)
2] Scope Resolution Operator (::)
3] Size of Operator (sizeof)
4] Conditional Operator (?:)

Although, the cementics of an operator can be operated, we cannot change its syntax, the grammatical rules that govern its use as the number of operands, precedance & associativity. Ex: the multiplication operation will enjoy the higher precedence then the addition operator. Remember when the operator is overloaded its meaning is not lost. For instance ~~when an~~ the operator (+) which has been overloaded to add two vectors or to add two strings, can still be used to add two integers.

Defining Operator Overloading: To define an additional tasks to an operator we must specify what it means in the relation to the class to which the operator is applied. This is done with the help of special function

called operator function which describes the task. The general form of operator function is:

<return type> <class name> :: operator <opr> (argument).
{
_____;
_____;
_____;
}.

where, <return type> is type of the value return by the specified operation & <opr> is the operator being overloaded Operator <opr> is the function name where "operator" is the keyword.

Operator functions must be either member functions or friend functions. A basic difference between them is that a friend function will have only one argument for unary operators & two for binary operators, while a member function has no arguments for unary operators & only one for binary operators. This is because the object is used to invoke the member function is passed implicity & therefore is available for member function. This is not the case with friend function. Arguments can be passed by value or by reference.

P28]

→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
using namespace std;
class Result
{
  int rno;
  double mrlc;
public:
  Result()
  {
```

```
    rno=0;
    mrk=0.0;
  }
  Result (Result &t)
  {
    rno=t.rno;
    mrk=t.mrk;
  }
  ~Result(){}
  friend istream &operator >> (istream &in, Result &t)
  {
    cout<<"\nRoll No.:";
    in >>t.rno;
    cout<<"\n Marks:";
    in >>t.mrk;
    return in;
  }
  friend ostream &operator << (ostream & out , Result &t)
  {
    out<<"\nRoll No:"<<t.rno << "\n Marks:"<<t.mrk;
    return out;
  }
};
void main ()
{
  Result d;
  cin>>d;
  cout<<d;
}.
```

P29] Matrix Operations using +, -, x

→

```
#include<stdio.h>
#include<stdlib.h>
#include<iostream>
```

DATE 06 08 2024

```
#include <string.h>
using namespace std;
class Matrix
{
   int x[3][3];
public:
   Matrix();
   Matrix (Matrix &);
   ~Matrix ( ) { }
   friend istream & operator >> (istream &, Matrix &);
   friend ostream & operator << (ostream &, Matrix &);
   Matrix operator + (Matrix &);
   Matrix operator - ( Matrix &);
   Matrix operator * ( Matrix &);
};

void main ()
{
   Matrix p, q, r;
   cout << " In Matrix 1:In:";
   cin >> p;
   cout << " In Matrix 2:In";
   cin >> q;
   r= p+q;
   cout << p << "In";
        << q << "In";
        << r ;
}

Matrix :: Matrix ()
{
   int i, j;
   for (i=0; i<3; i++)
```

```
        for(j=0; j<3; j++)
            x[i][j]=0;
    }
}
Matrix :: Matrix(Matrix &t)
{
    int i,j;
    for (i=0; i<3; i++)
    {
        for (j=0; j<3 ; j++)
            x[i][j] = t.x[i][j];
    }
}
istream &operator >> (istream &in , Matrix &t)
{
    int i,j;
    cout <<"\n Matrix Data";
    for (i=0; i<3; i++)
    {
        for (j=0; j<3; j++)
        {
            in >> t.x[i][j];
        }
    }
    return in;
}
ostream & operator << (ostream &out, Matrix &t)
{
    int i,j;
    cout << "\n Matrix \n";
    for (i=0; i<3; i++)
    {
        for(j=0; j<3; j++)
            out <<" " << t.x[i][j];
        out <<"\n";
    }
    return out;
}
```

CLASSMATE

DATE

```
Matrix Matrix :: operator + (Matrix &t)
{
   Matrix d;
   int i, j;
   for (i=0; i<3; i++)
   {
      for (j=0; j<3; j++)
         d.x[i][j] = x[i][j] + t.x[i][j];
   }
   return d;
}
Matrix Matrix :: operator - (Matrix &t)
{
   Matrix d;
   int i, j;
   for (i=0; i<3; i++)
   {
      for (j=0; j<3; j++)
         d.x[i][j] = x[i][j] - t.x[i][j];
   }
   return d;
}
Matrix Matrix :: operator * (Matrix &t)
{
   Matrix d;
   int i, j, k;
   for (i=0; i<3; i++)
   {
      for (j=0; j<3; j++)
      {
         for (k=0; k<3; k++)
            d.x[i][j] += (x[i][k] * t.x[k][j]);
      }
   }
   return d;
}
```

classmate

PAGE

DATE 08 08 2024

P30] →

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
using namespace std;
class Number
{
   int no;
 public:
    Number()
    {
       no = 0;
    }
    Number(int a)
    {
       no = a;
    }
    Number(Number &t)
    {
      no = t.no;
    }
   ~Number() { }
   friend istream &operator >> (istream &in, Number &t)
   {
      cout << "\n No:";
      in >> t.no;
      return in;
   }
  friend ostream &operator << (ostream &out, Number &t)
  {
     out << "\n No:" << t.no;
     return out;
  }
  Number operator + (int a)
```

```
void main()
{
   Number ic;
   cin >> ic;
   cout << ic;
}
```

classmate PAGE

DATE

```
{
    Number t;
    t.no = no+a;
    return t;
}
Number operator - (int a)
{
   Number t;
   t.no = no-a;      } Number t(no-a); (or)
   return t;
}
Number operator + (Number &k)
{
    Number t(no+k.no);
    return t;
}
Number operator - (Number &k)
{
    Number t(no-k.no);
    return t;
}
Void operator -()
{
   no = -no;
} // call from main => -p
Number operator += (int a)
{
    no= no+a;
    return *this;
} // p+= 3
Number operator -= (int a)
{
    no= no-a;
    return *this;
} // p-=3
```

```
Void main()
{
  Number p,q,r;
  cin>>p>>q;
  r = p+q;
  cout<<r;
}
```

PAGE

DATE

```
Number operator ++ ()  // pre increment.
{                                    q = ++p
    no++;
    return *this;
}
Number operator ++ (int)  //post increment.
{                                    q = p++
    Number t(no++);
    return t;
}
};

void main()
{
    Number k;
    cin >> k;
    cout << k;
}
void main()
{
    Number p, q, r;
    cin >> p >> q;
    r = p+q;
    cin << p << q << r;
    return;
}
void main()
{
    Number p, q, r;
    cin >> p >> q;
    cout << p << q;
    r = p+3;
    cout << r;
    p+ = 4;
```

DATE 12 08 2024

```
    cout <<p;
    q = ++p;
    cout <<p<<q;    q = p++;  cout <<p<<q;
    -p;
    cout << p;
    -p;
    cout <<p;
}
```

31] repeated

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
using namespace std;
class Result
{
   int rno;
   double mrk;
  public:
    Result()
    {
        rno = 0;
        mrk = 0.0;
    }
    Result(Result &t)
    {
        rno = t.rno;
        mrk = t.mrk;
    }
   ~Result(){}
   friend istream & operator >> (istream &in , Result &t)
   {
        cout << "\n Roll. No: ";
        cin >> rno;
```

```
    cout << "In Marks:";
    cin >> mrk;
    return in;
}
friend ostream &operator << (ostream &out, Result &r)
{
   out << "InRoll.No:" << r.rno << "InMarks:" << r.mrk;
   return out;
}
int operator > (int n)
{
   return (rno>n);
}
int operator< (int n)·
{
   return (rno<n);
}
int operator (Result &t)
{
   return (rno >t.rno);
}
int operator (Result &t)
{
   return (rno<t.rno);
}
Result operator +(double d)
{
   Result a;
   a.rno = rno;
   a.mrk = mrk+d;
   return a;
}.
Result operator -(double d)
{
```

DATE

```
    Result a;
    a.rno = rno;
    a.mrk = mrk-d;
    return a;
  }
};

void main()
{
  Result r;
  cin >>r;
  cout<<r;
  if (r.isMore(13))  (r > 13)
     cout << "In > 13";
  else
     cout << "In <=13";
}

void main()
{
  Result r, s;
  cin >>r >> s;
  cout <<r <<s;
  if (r >s)
     cout <<r;
  else
     cout <<s;
  s = s+3.2;
  cout <<s;
}

//array of object of type result.
void main()
{
```

PAGE

classmate

```
    Result *r = new Result [5];
    for (int i=0 ; i<5 ; i++)
        cin >>r[i];
    for( int i=0; i<5 ;i++)
        cout<< r[i];
    delete[]r;
}

//Sorting of array on Roll No. basis. (selection sort)
void main ()
{
    Result r[5];
    for (int i=0; i<5 ; i++)
        cin >> r[i];
    for (int i=0 ; i<5 ; i++)
        cout <<r[i];
    int j, pos; Result tmp;
    for(int i=0; i<4; i++)
    {
        pos = i;
        for (int j=i+1 ; j<5 ; j++)
        {
            if (r[j] <r[pos])
                pos=j;
        }
        if (i!=pos)
        {
            tmp= r[i];
            r[i] =r[pos];
            r[pos]= tmp;
        }
    }
    for (int i=0; i<5; i++)
        cout << r[i];
}
```

DATE 17/02/2024

Type Conversion:-

	Source	Destination
1]	Basic	User Defined Type (using constructor fn)
2]	User defined (By defining operator <dest>())	Basic Type
3]	User defined a] By defining operator <dest>()	User defined b] By defining constructor fn with source as argument.

Syntax for Operator Function:

```
operator <dest>()
{ ---------- ;
  return <dest_type_value>;
}
```

Characteristics:

- This are special purpose functions.
- Always to be defined in public section of class.
- This functions don't have returning type not even void, but return statement is must.
- Name of the function is returning type of the function.
- This functions can't be overloaded as functions don't have argument.

P→ // User defined to Basic type conversion.

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
using namespace std;
class Result
{
    int no;
    double mrks;
```

DATE

```
public:
    Result()
    {
        no = 0;
        mrk = 0.0;
    }
    Result(int a, double b)
    {
        no = a; mrk = b;
    }
    Result(Result &t)
    {
        no = t.no;
        mrk = t.mrk;
    }
    ~Result() { }
    operator int()
    {
        return no;
    }
    operator double()
    {
        return mrk;
    }
    friend istream &operator >> (istream &in, Result &t)
    {
        cout << "\nRoll. No:";
        in >> t.no;
        cout << "\nMarks:";
        in >> t.mrk;
        return in;
    }
    friend ostream &operator << (ostream &out, Result &t)
    {
```

```
    out << "\nRoll No:" << t.no << "\n Marks:" << t.mrk;
    return out;
  }
};

void main()
{
   Result a;
   cin >>a;
   cout << a;
   int b=a;    // to int
   double c=a; // to double.
   cout << "\n No:" << b<<"\nMarks:" << c;
}.
```

P→

```
// Type conversion from class to class.
#include <stdio.h>
#include <stdlib.h>
# inslude <iostream>
# include <string.h>
using namespace std;
class Marks
{
   int rno;
   double mrk[5];
 public:
   Marks();
   ~Marks() {}
   void setData();
   void display();
   int total();
   operator int()
   {
      return rno;
   }
```

```
    operator double()
    {
        double avg = double (total())/5;
        return avg;
    }
};

class Result
{
    int rno;
    double avg;
public:
    Result() { rno=0 ; avg=0.0;}
    Result (Mark &a)
    {
        rno = a;
        avg = a;
    }
    ~Result(){}
    void display()
    {
      cout << "\n Roll NO: " << rno << "\n Avg: " << avg;
    }
};

Marks :: Marks()
{
    int i=0;
    rno=0;
    for (i=0; i<5; i++)
        mrk[i]=0;
}
int Marks :: total()
{
```

} type conversion from marks to int/double.

```
    int i = 1, total = mrk[0];
    while (i<5)
        total += mrk[i++];
    return total;
}
void Marks :: setData()
{
    cout << "\n Roll No:";
    cin >> rno;
    cout << "\n Marks:";
    for (int i=0; i<5; i++)
        cin >> mrk[i];
}
void Marks :: display()
{
    cout << "\n Roll No:" << rno << "\nMarks:";
    for (int i=0; i<5; i++)
        cout << " " << mrk[i];
}

int main()
{
    Marks m;
    m.setData();
    m.display();
    Result r(m);
    r.display();
}
```

P→

```
//
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
```

```
using namespace std;
class Result
{
  int rno;
  double avg;
 public:
  Result () {rno=0; avg=0.0;}
  Result (int a, double b)
  {
     rno=a;
     avg=b;
  }
  Result (Result &t)
  {
     rno=t.rno;
     avg=t.avg;
  }
  ~Result () {}
  void display ()
  {
    cout<<"\n Roll No:"<<rno<<"\n Avg:"<<avg;
  }
};
class Marks
{
  int rno;
  int mrlc[5];
 public:
  Marks ();
  ~Marks () {}
  void setData ();
  void display ();
  int total ();
  Operator int ()
```

```
    {
        return rno;
    }
    operator double()
    {
        double avg = double (total())/5;
        return avg;
    }
    Operator Result()
    {
        int a = *this;    // marks to int type conversion
        double b = *this;    // calls operator double definition.
        Result r(a,b);
        return r;
    }
};

// class to class conversion.
Marks :: Marks()
{
    rno=0;
    for (int i=0; i<5; i++)
        mrk[i]=0;
}
int Marks :: total()
{
    int i=1, tot=mrk[0];
    while (i<5)
        tot += mrk[i++];
    return tot;
}
void Marks :: setData()
{
    cout << "\n Roll No: ";
```


classmate

DATE 21/02/2024

```
    cin >> rno;
    cout << "\n Marks : ";
    for (int i=0; i<5; i++)
        cin >> mrk[i];
}
void Marks :: display()
{
    cout << "\n Roll No: " << rno;
    cout << "\n Marks \n";
    for (int i=0; i<5; i++)
        cout << " " << mrk[i];
}

int main()
{
    Marks m;
    m.setData();
    m.display();
    Result r;
    r=m;
    r.display();
}
```

- **Inheritance : Extending Class :-**

Reusability is yet another important feature of OOP. It is always nice if we could reuse something that already exist rather than trying to create the same all over again. It would not only save time & money but also reduce frustration & increase reliability, for instance the reuse of a class that has already been tested, debugged & used many times can save us the effort of developing & testing the same again.

Fortunately, the C++ strongly supports the concept of reusability. The C++ classes can be reused in several

ways. Once a class has been written & tested it can be adapted by other programmers to suit there requirements. This is basically done by creating new classes, reusing the properties of existing one's, The mechanism of deriving a new class from an old one is called inheritance. The old class is reffered to as base class & the new one is called the derived class.

The derived class inherits some ~~of~~ or all of the ~~tra~~ trates from the base class. A class can also inherit properties from more than one class or from more than one level. A derived class with only the base class is called single inheritance & one with several base class is called multiple inheritance. On the other hand the trates of one base class may be inherited by more than one class. This process is known as hierathial inheritance. The mechanism of deriving the class from another derived class is known as multi-level inheritance.

The following diagram shows various form of inheritance that can be used for writing extensible Programs. The direction of arrow indicates the direction of inheritance.

1] A
↓
B
Single Inheritance

2] A
↓
B
↓
C
Multilevel Inheritance

3] A B
↓
C
Multiple Inheritance

4] A
B C
Hierarchical Inheritance.

5] a] A
B C
D
Hybrid Inheritance

b] A
B C
D

Syntax:

```
class <derived class> : [<derivation type>] <list_of_base_classes>
{
        ____________;
        ____________;
        ____________;
};
```

Derivation Type:

1] private (by default)

2] public.

Base Class	Derivation Type	Derived Class
• private	private	NA
private	public	NA
• protected	private	private
protected	public	protected
• public	private	private
public	public	public.

P→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <string.h>
using namespace std; //implementation of single inheritance with
class Student //Base class.        public derivation.
{
   int rno;
   char nm[15];
 public:
   void setData()
   {
      cout << "\nRollNo:";
      cin >> rno;
      cout << "\nName:";
      cin >> nm;
   }
```

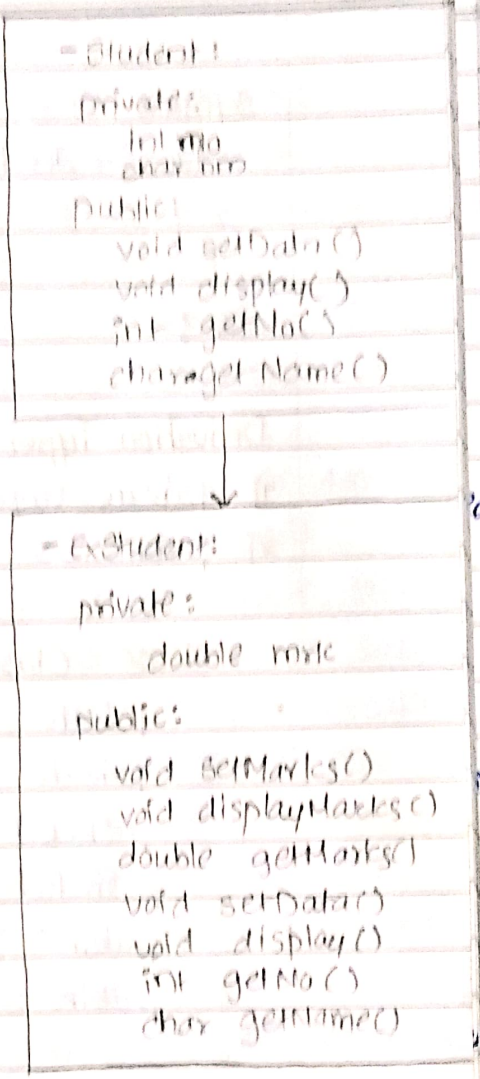
Block Diagram

DATE

```
    void display()
    {
        cout << "\nRoll No:" << rno;
        cout << "\nName: " << nm;
    }
    int getNo() { return rno; }
    char * getName() { return nm; }
};

class ExStudent : public Student   // Derived class.
{
    double mrk;
  public:
    void setMarks()
    {
        cout << "\nMarks:";
        cin >> mrk;
    }
    void displayMarks()
    {
        cout << "\nMarks" << mrk;
    }
    double getMarks()
    {
      return mrk;
    }
};

void main()
{
    Student s;
    s.getData();
    s.display();
    ExStudent d;
```



PAGE

```
    d.setData();
    d.setMarks();
    d.display();
    d.displayMarks();
}
```

P→ //Single inheritance of ~~cir~~ with private derivation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <iostream>
using namespace std;
class Student
{
    int rno;
    char nm[15];
  public:
    void setData()
    {
        cout<<"\nRoll No:";
        cin>>rno;
        cout<<"\nName:";
        cin>>nm;
    }
    void display()
    {
        cout<<"\nRoll No:"<<rno<<"\nName:"<<nm;
    }
    int getNo()
    {
        return rno;
    }
    char *getName()
    {
```

```
    return nm;
  }
};

class ExStudent : Student
{
    double mrk;
  public:
    void setMarks()
    {
        cout << "\nEnter Marks";
        cin >> mrk;
    }
    void displayMarks()
    {
        cout << "\nMarks:" << mrk;
    }
    double getMarks()
    {
        return mrk;
    }
    void input()
    {
        setData();
    }
    void output()
    {
        display();
    }
};

int main()
{
    ExStudent d;
```

- Student:

```
private:
    int rno;
    char nm[15];
public:
    void setData()
    void display()
    int getNo()
    char * getName()
```

- ExStudent:

```
private:
    double mrk;
    void setData()
    void display()
    int getNo()
    char *getName()
public:
    void setMarks()
    void displayMarks()
    double getMarks()
    void input()
    void output()
```

DATE 27/02/2024

```
    d.input();
    d.setMarks();
    d.output();
    d.displayMarks();
}
```

P→ // Multilevel inheritance.

```
#include<stdio.h>
#include<stdlib.h>
#include<iostream>
using namespace std;
class Number
{
  protected:
    int no;
  public:
    Number()
    {
      no=0;
    }
    ~Number(){}
    void setNo()
    {
      cout<<"\nRoll No:";
      cin>>no;
    }
    void displayNo()
    {
      cout<<"\nRoll No:"<<no;
    }
};
class Name: public Number
{
  protected:
```

```
    char nm[15];
public:
    Name(): Number()
    {
        nm[0] = '10';
    }
    ~Name(){}
    void setName()
    {
        cout <<"\nName:";
        cin >> nm;
    }
    void displayName()
    {
        cout <<"\nName:"<<nm;
    }
};
class Marks : public Name
{
    protected:
        double mrk;
    public:
        Marks(): Name()
        {
            mrk = 0.0;
        }
        ~Marks(){}
        void setMarks()
        {
            cout <<"\nMarks:";
            cin >>mrk;
        }
        void displayMarks()
        {
```

```
        cout <<"\nMarks:" <<mrk;
    }
};

void main()
{
    Number a;
    a.setNo();
    a.displayNo();
    Marks b;
    b.setNo();
    b.setName();
    b.setMarks();
    b.displayNo();
    b.displayName();
    b.displayMarks();
}
```

P→

```
class Number
{
    protected:
        int no;
    public:
        Number()
        {
            no=0;
        }
        ~Number(){}
        void setNO()
        {
            cout <<"\nRoll No:";
            cin >> no;
        }
        void displayNO()
        {
            cout <<"\nRoll No" <<no;
        }
};
```

```
class Name: public Number
{
    protected:
        char nm[15];
    public:
        Name(): Number()
        {
            nm[0] = '\0';
        }
        ~Name(){}
        void setName()
        {
            setNo();
            cout << "\nName:";
            cin >> nm;
        }
        void displayName()
        {
            displayNo();
            cout << "\nName:" << nm;
        }
};
class Marks: public Name
{
    protected:
        double mrk;
    public:
        Marks(): Name()
        {
            mrk = 0.0;
        }
        ~Marks(){}
        void setMarks()
        {
```

```
        setName();
        cout << "In Marks:";
        cin >> mrk;
    }
    void displayMarks()
    {
        displayName();
        cout << "In Marks:" << mrk;
    }
};

void main()
{
    Number a;
    a.setNo();
    a.displayNo();
    Marks b;
    b.setMarks();
    ~~else~~ b.displayMarks();
}
```

P→

```
class Number
{
  protected:
    int no;
  public:
    Number()
    {
        no=0;
    }
    ~Number(){}
    void setData()
    {
        cout << "In No:";
```

```
    cin >>no;
  }
  void display()
  {
    cout <<"\nNo:" <<no;
  }
};
class Name : public Number
{
  protected:
    char nm[15];
  public:
    Name(): Number()
    {
      nm[0]='\0';
    }
   ~Name(){}
    void setData()
    {
      cout <<"\nName:";
      cin >> nm;
    }
    void display()
    {
      cout <<"\nName:" <<nm;
    }
};
class Marks : public Name
{
  protected:
    double mrk;
  public:
    Marks(): Name()
    {
```

```
    mrk = 0.0;
  }
  ~ Marks() { }
   void setData()
   {
      Cout << "\n Marks: " ;
      cin >> mrk;
   }
   void display()
   {
      Cout << "\n Marks : " << mrk;
   }
};
void main ()
{
   Number k;
   k. setData();
   k. display();
   Mark a;
   a. Number :: SetData();
   a. Name :: SetData();
   a. setData();
   a. Number :: display();
   a. Name :: display();
   a. display();
}

P →  class Number
{
   protected:
      int no;
   public:
     Number()
     {
```

```
    no = 0;
  }
  ~Number(){}
  void setData()
  {
     cout<< "\n No:";
     cin >> no;
  }
  void display()
  {
     cout << "\n No:" <<no;
  }
};
class Name : public Number
{
   protected :
      char nm[15];
   public:
      Name(): Number()
      {
         nm[0] = '\0';
      }
      ~Name(){}
      void setData()
      {
         Number :: setData();
         cout << "\n Name:";
         cin >>nm;
      }
      void display()
      {
         Number :: display();
         cout << "\n Name:" <<nm;
      }
};
```

DATE

```
class Marks : public Name
{
   protected :
       double mrk;
   public :
       Marks() : Name()
       {
          mrk = 0.0;
       }
       ~ Marks(){}
       void setData()
       {
          Name :: setData();
          cout << "\nMarks:";
           cin >> mrk;
       }
       void display()
       {
          Name:: display();
          cout << "\nMarks:" << mrk;
       }
};
void main()
{
   Number k;
    k . setData();
    k . display();
     Mark a;
    a. setData();
     a. display();
}
```

DATE 09 08 2024

```
// Hierarchial Inheritance.
# define N 5
# include <stdio.h>
# include <stdlib.h>
# include <iostream>
using namespace std;
class Array
{
   protected:
      int arr[N];
   public:
      Array ();
     ~Array (){};
      void setData();
      void display ();
};
Array :: Array()
{
   for(int i=0; i<N; i++)
      arr[i]=0;
}
void Array :: setData()
{
   cout <<"\nData In";
   for (int i=0; i<N; i++)
      cin >>arr[i];
}
void Array :: display()
{
   cout <<"\nData\n";
   for (int i=0; i<N; i++)
      cout<<" "<<arr[i];
}
```

PAGE

```
class Search : public Array
{
    public :
        Search (): Array ()
        { }
        ~ Search (){}
        int isordered ();
        int bsearch( int sv);
        int ssearch (int sv);
};
int Search :: isordered ()
{   int i;
    for (int i=1; i<N; i++)
    { if ( arr[i] < arr[i-1])
        break;
}  }
    if (i==N)
        return 1;
    else
        return 0;
}
int search :: ssearch(int sv)
{
    int i=0;
    while (i<N)
    {
        if (arr[i]==sv)
            break;
        i++;
    }
    if (i==N)
        return 0;
    else
        return 1;
}
```

DATE

```
int Search :: bsearch(int sv)
{
   if (isordered()==0)
      return 0;
   int l=0, r=N-1, mid;
   do
   {
      mid = (l+r)/2;
      if (arr[mid]==sv)
         break;
      if (sv<arr[mid])
         r = mid-1;
      else
         l = mid+1;
   } while (l<=r);
   if (l<=r)
      return 1;
   else
      return 0;
}

class Sort : public Array
{
   public:
      Sort(): Array(){}
      ~Sort(){}
      void ssort();
      void bsort();
};
void Sort :: ssort()
{
   int i=0, j,pos, tmp;
   while (i<N-1)
   {
```

PAGE

classmate

```
    pos = i;
    for (j= i+1; j<N ; j++)
    {
        if (arr[j]< arr [pos])
            pos=j;
    }
    if (i!=pos)
    {
        tmp= arr[i];
        arr[i] = arr[pos];
        arr[pos] = tmp;
    }
    i++;
  }
}
void Sort :: bsort()
{
   int i=0, j, tmp;
   for (i=0; i<N-1 ; j++)
   {
       for (j=0; j<N-i; j++)
       {
           if (arr[j] > arr[j+1])
           {
               tmp = arr[j];
               arr[j]= arr[j+1];
               arr[j+1] = tmp;
           }
       }
   }
}

void main()
{
```

```
search p; int res;
p.setData();
p.display();
int flg = p.isordered();
if (flg==1)  // data is ascending.
    res = p.bsearch(42);
else
    res = p.ssearch(42);
if (res==1)
    cout<<"\n Found";
else
    cout<<"\n Not found";
sort q;
q.setData();
q.display();
q.ssort();
q.display();
}
```

- Multiple Inheritance :-

P→
```
//Multiple Inheritance.
#include <stdio.h>
#include <iostream>
#include <stdlib.h>
#
class sem1
{
  protected:
    int t1,t2,o1;
  public:
    sem1()
    {
      t1=t2=o1=0;
    }
```

DATE

```
    ~Sem1(){}
    void setSem1()
    {
      cout<<"In Theory Marks In";
      cin>>t1>>t2;
      cout<<"In Oral Marks: In";
      cin>>o1;
    }
    void display()
    {
      cout<<"In Sem1 Theory In";
      cout<<"~~In t1<<"  "<<t2;
      cout<<"Oral In<<o1<<"  "<<o2;
    }
};
Class Sem2.
{
  protected:
    int t1, t2, o1, o2;
  public:
    sem2()
    {
      t1=t2=o1=o2=0;
    }
    ~sem2(){}
    void setsem2()
    {
      cout<<"In Theory Marks In";
      cin>>t1>>t2;
      cout<<"In Oral Marks In:";
      cin>>o1>>o2;
    }
    void display()
    {
```

DATE

```
    cout << "In Sem2 Theory: In";
    cout << t1 << "  " << t2;
    cout << "In Oral In" << o1 << "  " << o2;
  }
};
class Result : public sem1, public sem2
{
  protected:
    int ttot, otot;
  public:
    Result(): sem1(), sem2()
    {
      ttot = otot = 0;
    }
  ~Result(){}
    void setData()
    {
      ttot = sem1::t1 + sem1::t2 + sem2::t1 + sem2::t2;
      otot = sem1::o1 + sem2::o1 + o2;
    }
    void display()
    {
      sem1::display();
      sem2::display();
      cout << "In Theory Total:" << ttot;
      cout << "In Oral Total:" << otot;
    }
};
void main()
{
  Result r;
  r.setsem1();
  r.sem1::display();
  r.setsem2();
```

PAGE

classmate

DATE 10 09 2024

```
    r.sem2 :: display();
    r.setData();
    r.display();
}
```

P→ // Hybrid Inheritance

```
#Stdio.h>
#inclade <stdlib.h>
#include <iostream>
using namespace std;
class A
{
  protected:
    int no;
    char nm[15];
  public:
    A() { no=0; nm[0] = '\0';}
    ~A() { }
    void setData()
    {
        cout <<"\nRoll No:";
        cin >>no;
        cout <<"\n Name:";
        cin >> nm;
    }
    void display()
    {
       cout <<"\nRoll No: "<< no <<" Name: "<< nm;
    }
};
class B : public A
{
   protected :
      int mrk;
```

```
public:
  B(): A() {mrk=0; }
 ~B() {}.
  void setData()
  {
     A :: setData();
     cout << "In Marks:";
     cin >> mrk;
  }
  void display()
  {
     A:: display();
     Cout << "In Marks :" << mrk;
  }
};
class C
{
  protected:
     int smrk;
  public:
     C() { smrk=0; }
    ~C() {}
     void setData()
     {
        cout << "InSports Marks:";
        cin >> smrk;
     }
     void display()
     {
        cout <<"In Sports Marks:" << smrk;
     }
};
class D: public B, public C
{
```


classmate

DATE

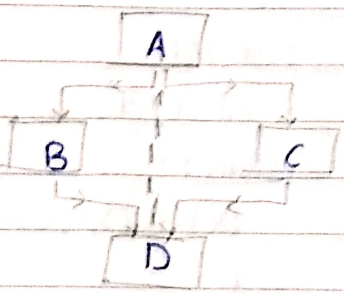
```
protected:
  int total;
public:
   D():B(), C()
   {
     total = 0;
   }
  ~D(){}
   void setData()
   {
     B :: setData();
     C :: setData();
     tot = mrk+smrk;
   }
   void display()
   {
     B :: display();
     C :: display();
     cout<<"In Total Marks:" <<total;
   }
};
void main()
{
   D obj;
   obj.setData();
   obj.display();
}
```

- Virtual Base Class

```
class A
{
};
class B: public A
```

DATE

```
{
};
class C : public A
{
};
class D : public B, public C
{
};
```



Consider above inheritance which involves 3 kinds of inheritences namely, multilevel, multiple & heirarchial. The class D has two direct base classes B & C which themselves have a common base class A & the D inherits the traits of A via to separate part. It can also inherit directly as shown with broken lines. The A is sometime ~~de ffe~~ refered to as indirect base class. The inheritance by D as shown above might make some problems. All the public & protected members on A are inherited to D twice, first via B & again via C, this means D would have doplicate sets of members inherited from A. This inheritance ambiguti & should be avoided.

The duplication of inherited members due to these multiple paths can be ~~constat~~ avoided by marking common base class (Ancestor Class) as virtual base class while declaring direct or intermediate base classes as shown below:

```
class A
{
};
class B : public virtual A.
{
};
class C : public virtual A
{
};
```

PAGE

classmate

class D public B, public C.
{
};

When class is made virtual base class C++ takes necessary care to see that only one copy of that class is inherited regardless of how many inheritance paths exist between virtual base class & derived class.

P→ //Mutilevel Inheritance. (Runtime Polymorphism)

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class A
{
  int a;
 public:
   void set()
   {
      cout << "\nAa:" ;
      cin >>a;
   }
};
class B : public A.
{
   int b;
   public:
     void set()
     {
        cout << "\nB b:";
        cin >> b;
     }
     void setdisplay()
     {
```

```
void display()
{
  cout <<"\nAa:" <<a;
}
```

```
        cout << "\nBb:" <<b;
    }
};
class C : public B.
{
    int c;
    public:
        void set()
        {
            cout<< "\nCc:";
            cin >> c;
        }
        void display()
        {
            cout << "\nCc:" << c;
        }
};
void main()
{
    A obja;
    obja.set(); // A's
    obja.display(); // A's
    B objb;
    objb.set(); // B's
    objb.display; // B's
    objb.A :: set(); // A's
    objb.A :: display(); // A's
    objb.display();
    C objc;
    objc.set(); // C's
    objc.display(); // C's
}
// Use of base pointer to access member function.
void main()
```

DATE

```
{
    A obja;
    A *aptr = &obja;
    aptr → set();        //A's
    aptr → display();    //A's
    B objb;
    aptr = &objb;
    aptr → set();        //A's
    aptr → display();    //
    ((B*)aptr) → set();      //B's
    ((B*)aptr) → display();
    C* cptr = new C;
     cptr → set();
     cptr → display();
}.
```

- Pointer to Derived Classes:-

We can use pointers not only to the base objects but also to the object of derived classes. Pointers to a object of a base class are type compatible with pointers to objects of a derived class. Therefore a single pointer variable can be made to point to objects belonging to derived classes. Consider a multilevel Inheritance where A is base class & B is a derived class from classA. classC is derived from B. Consider the following declaration:

```
A* aptr;  // Pointer to class A
A obja;   // Base class object.
B objb;   // Derived class object.
aptr = &obja; // Aptr points to obj A.
```

We can make 'aptr' to point to an object objb as follows: aptr = &objb.

This is perfectly valid with C++ because objb is an object derived from class A. However there is a problem using aptr to access the public

member of derived class B. Using aptr we can only access those element which are inherited from A & not the membes originaly belonging to B.

In case, member of B has same name as one of the members of A, then any me reference to that member by aptr will always access the base class member.

Although C++ permits a base pointer to point to any object derived from base. The pointer can't be directly used to access all members of derived class. We may have to use another pointer declare as pointer to derived type.

Virtual functions:

As mentioned earlier, polymorphism refers to the property by which objects belonging to different classes are able to respond to the same message but in different forms. An essential requirement of polymorphism is therefore the ability to refer to objects without any regards to there class. Here we use the pointer to the base class to refer to all the derived objects but we just discovered that the base pointer even when it is made to contain address of derived class always execute the functions in base class. The compiler simply ignores content of pointer & chose member function that matches type of pointer. How do we then achieve polymorphism?. It is achieved using what is known as virtual functions.

When we use the same function name in both the base & derived classes, the function in base class is declared as virtual using the keyword "virtual" preceeding its normal declaration. When a function is made virtual, C++ determines which function to use on runtime, based on type of object pointed to by the base

pointer, rather than the type of the pointer. Thus by making the base pointer to point to different objects we can execute different versions of virtual function.

P-7

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class A
{
   int a;
   public:
virtual void set()
      {
         cout <<"\nAa:";
         cin >> a;
      }
virtual void display()
      {
         cout <<"\nAa:" << a;
      }
};
class B: public A.
{
   int b;
   public:
      void set()
      {
         cout <<"\nBb:";
          cin >> b;
      }
      void display()
      {
         cout <<"\nBb:" << b;
      }
};
```

classmate

PAGE

DATE

```
class C : public B
{
  int c;
  public:
    void set()
    {
        cout << "\n Cc:";
        cin >> c;
    }
    void display()
    {
        cout << "\n Cc:" << c;
    }
};

void main()
{
    A * aptr;
    A obja;
    aptr = &obja;
    aptr → set();
    aptr → display();          // A;
    B objb;
    aptr = &objb;
    aptr → set();
    aptr → display();          // B;
    C objc;
    aptr = &objc;
    aptr → set();
    aptr → display();          // C;
}
```

One important point to remember is that we must access virtual function through the use of a pointer declared as a pointer to the base class. Remember run-time

PAGE

DATE

polymorphism is achieved only when a virtual function is accessed through pointer to a base class.

Rules for Virtual Class:

1) When virtual functions are created for implementing late binding we should observe some basic rules that satisfy the compiler requirements.
   - The virtual ~~et~~ functions must be member of some class.
   - They cannot be static members.
   - They are accessed by using object pointers.
   - A virtual function can be a friend of another class.
   - A virtual function in a base class must be defined, ~~e~~venthough it may not be used.
   - The prototypes of the base class ~~fun~~ version of virtual function & all derived class versions must be ~~e~~ identical. If the two functions with same function name have different prototypes, C++ consider them as overloaded functions. & the virtual mechanism is ignored.
   - We cannot have virtual constructors but we can have virtual destructors.
   - While a base pointer can point to any type of the derived object, the reverse is not true. that is to say we cannot use pointer to derived class to access the object of base type.
   - When a base pointer points to a derived class incrementing or decrementing it will not make it to point the next object of derived class.
   - It is ~~increpad~~ incremented / decremented only relative to its base type. Therefore we should not use this method to move pointer to next object.
   - If virtual ~~pointer~~ function is defined in base class, need not be necessary redefined in derived class

DATE 19/09/2024

In such cases calls will invoke base function.

Pure Virtual Function :- It is normal practice to declare a fn virtual inside the base class & redefine it in the derived classes. The fn inside the base class is seldom used for performing any task. It only serves as a place holder. Consider following multilevel inheritance;

dim
↓
rect
↓
trang

we have not defined any object of class dim & therefore the fn area in the base class can be defined empty. Such functions are called do-nothing functions. Such functions may be defined as follows:

virtual void area = 0;

Such functions are called pure virtual function. A pure virtual fn is a fn declared in base class that has no defination related to base class. In such cases the compiler each derived class to either ~~decla~~ derive or redeclare it as pure virtual fn. Remember that the class containing pure virtual fn cannot be used to declare any objects of its own, Such classes are called abstract base classes. The main objective of an abstract base class is to provide some traits to the derived classes & to create base pointer required for achieving runtime polymorphism.

P→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
class dim
{
  protected:
```

```
    int w, h;
  public:
    dim(){ w=h=0;}
    dim(int a, int b)
    {
       w=a; h=b;
    }
    ~dim(){}
    void setData()
    {
       cout<<"\nWidth:";
       cin>>w;
       cout<<"\n Height:";
       cin>>h;
    }
    void display()
    {
       cout<<"\nWidth:"<<w<<"\n Height:"<<h;
    }
    virtual void area()=0; // Pure virtual fn.
};
class rect : public dim
{
  public:
    rect(): dim()
    {}
    rect(int a, int b): dim(a,b);
    {}
    ~rect(){}
    void area()
    {
       cout<<"\n Rect Area:"<<(w*h);
    }
};
```

DATE

```
class trang: public rect
{
   public:
     trang(): rect() {}
     trang(int a, int b): rect (a, b) {}
     ~trang () {}
      void area()
      {
         cout << "\n Trang Area:" << (w*h)/2;
      }
};
void main ()
{
    dim *ptr;
    ptr = new rect (10,5);
     ptr → display();
     ptr → area();
     ptr = new trang (10, 15);
     ptr → display ();
     ptr → area ();
}
void main()
{
   dim *ptr;
   cout << "\n 1. Rect \n 2. Trang \n 3. Exit \n Choice:");
   int opt;
   cin >> opt;
   if (opt>2)
      return;
   if (opt == 1)
      ptr = new rect();
   else
      ptr = new trang();
   ptr → setData();
```

PAGE

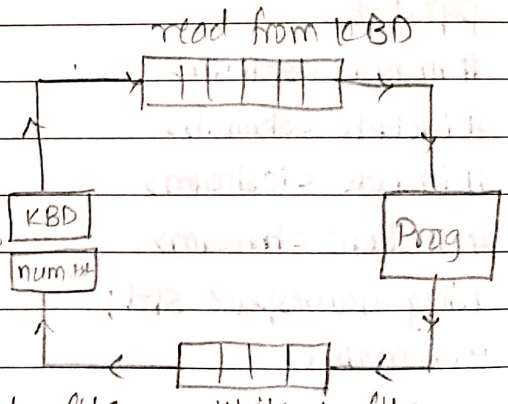
```
    ptr → display();
    ptr → area();
    delete ptr;
}
```

- File Handling in C++ :- Classes available in C++ to manuplute files.

i] ofstream ⇒ class object is used to write to file.
ii] ifstream ⇒ class object is used to read from file.
iii] fstream ⇒ class object is used to read/write from/to file.

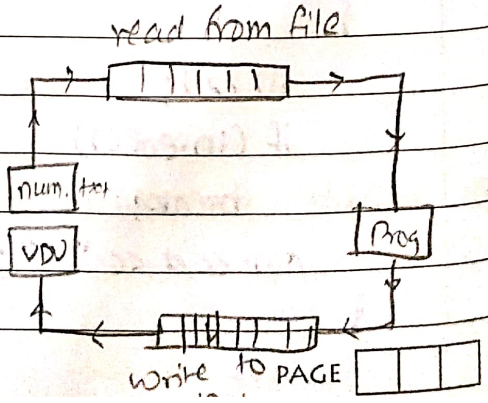
P] Program to create text file containing numbers.

→
```
#include <fstream>  #include <stdio.h>  #include <stdlib.h>
                                         #include <iostream>
void main()                              using namespace std;
{
    ofstream out("num.txt");
    cout << "\n Enter Nos:";
    int d;
    while(1)
    {
        cin >> d; // read from keyboard
        if(d==0)
            break;
        out << d << " "; // write to file
    }
    out.close();
}
```



P] Program to read file num.txt.

→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <fstream>
using namespace std;
```



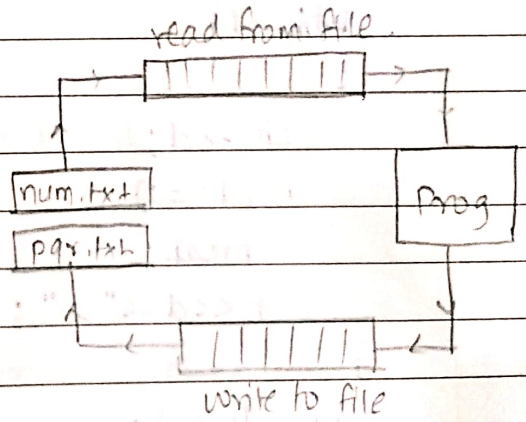
DATE

```
void main()
{
  ifstream in("num.txt");
  cout << "\n Enter Nos:";
  int d;
  while (1)
  {
     in >> d; // read from file
     if (in.eof())
        break;
     cout << d << "  "; // write to VDU.
  }
  in.close();
}
```

P] Write a program to copy source file num.txt with new name pqr.txt.

→

```
# include <stdio.h>
# include <stdlib.h>
# include <iostream>
# include <fstream>
using namespace std;
void main()
{
  ifstream in("num.txt");
  ofstream out("pqr.txt");
  int d;
  while(1)
  {
     in >> d; // read from source.
     if (in.eof())
        break;
     out << d << " "; // write to destination.
  }
```



PAGE

classmate

```
    in.close(); out.close();
}
```

P] Write a program to create a text file containing student record

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <fstream>
using namespace std;
void main()
{
   ofstream sout ("stu.txt");
   int no, mrk;
   char nm[15];
   cout << "In Student Data In";
   while (1)
   {
      cout << "In Roll No:";
      cin >> no;
      if (no==0)
         break;
      cout << "InName:";
      cin >> nm;
      cout << "In Marks:";
      cin >> mrk;
      sout << no << "  " << nm << "  " << mrk << "In";
   }
   sout.close();
   return;
}.
```

P] Write a program to read a text file containing student record

→

```
#include <stdio.h>
#include <stdlib.h>
```

DATE 21 09 2024

```
#include <iostream>
#include <fstream>
using namespace std;
void main()
{
   ifstream sin("stu.txt");
   int no, mrk;
   char nm[15];
   cout << "\n Student Data";
   while (1)
   {
      sin >> no >> nm >> mrk;
      if (sin.eof())
         break;
      cout << "\nRoll No:" << no << "\nName:" << nm << "\n Marks:" <<
            mrk;
   }
   sin.close();
}
```

H.P. Create a header file with name "stu.h" & implement class stu.

→
```
class stu // Save class with name "stu.h"
{
 int no;
 char nm[15];
 int mrk;
 public:
    stu()
    {
       no = mrk = 0;
       nm[0] = '\0';
    }
    ~stu() {}
```

```
    void setData(int a)
    {
        no=a;
        cout<<"\nName:";
        cin>>nm;
        cout<<"\nMarks:";
        cin>>mrk;
    }
    void display()
    {
        cout<<"\nRoll No.:"<<no
            <<"\n Name:"<<nm <<"\nMarks:"<<mrk;
    }
    int getNo(){return no;}
    int getMarks(){return mrk;}
};
```

P] Program to write "stu" class object to the binary file "stu.dat"

→

```
#include<stdio.h>
#include<stdlib.h>
#include<iostream>
#include<fstream>
#include "stu.h"
using namespace std;
void main()
{
    stu s;
    int no;
    ofstream sout;
    sout.open("stu.dat", ios::binary | ios::out);
    while(1)
    {
        cout<<"\nRoll No.:";
        cin>>no;
```

```
    if (no==0)
      break;
    s.setData (no)
    sout.write ((char *) &s, sizeof (stu));
  }
  sout.close();
}
```

P] Program to read "stu" class object from "stu.data" (binary file).

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <fstream>
#include "stu.h"
using namespace std;
void main()
{
  stu s;
  ofstream sin;
  sin.open ("stu.dat", ios::binary | ios::in);
  while(1)
  {
    sin.read((char *) &s, sizeof(stu));
    if (sin.eof())
      break;
    s.display();
  }
  sin.close();
}.
```

P] Create a binary file containing integer numbers.

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
```

DATE

```
#include <fstream>
#include
using namespace std;
void main()
{
   int no;
   ofstream out;
   out.open("num.dat", ios::binary | ios::out);
   while(1)
   {
      cout <<"\nEnter No:";
      cin>>no;
      if(no==0)
        break;
      out.write((char*) &no, sizeof(int));
   }
   out.close();
}
```

P] Program to read binary file containing numbers.

→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
#include <fstream>
using namespace std;
void main()
{
   int no;
   ifstream in;
   in.open("num.dat", ios::binary|ios::in);
   while(1)
   {
      in.read((char *) &no, sizeof(int));
      if(in.eof())
```

```
    break;
  cout<<ch<<" ";
  }
  in.close();
}
```

File Opening Modes: Here before we have used 'ifstream' & 'ofstream' constructor & function open() to create new files as well as to open the existing files. Remember that in both this methods we used only one argument that was file name. However this function can take two argument that is second one is for specifying the file mode. The general form of function open is:

<stream_object>.open ("filename", mode);

The second argument mode specifies the purpose for which the file is opened. The prototype of this class member function contain default values for the second argument & therefore they use the default values in the absence of the actual values. The default values are as follows:

ios::in ⇒ open for reading only.

ios::out ⇒ open for writing only.

File Mode Parameters:-

Parameter	Meaning.
ios::app	Append to end of file.
ios::ate	Go to end of file on opening.
ios::binary	Binary file
ios::in	open file for reading only
ios::out	open file for writing only
ios::nocreate	Open fails if file doesn't exist.
ios::noreplace	Open fails if file already exist
ios::trunc	Delete the content of file if it exist.

DATE

File Pointers & There Manipulation:-

Each file has to associated pointers known as the file pointers One of them is called the input pointer or get pointer & the other is called output pointer or put pointer. We can use these pointer to move through files while reading or writing. The input pointer is used for reading the contents of given file location. & output pointer is used for writing to given file location. Each time an input or output operation takes place the appropriate pointer is automatically advanced.

Default Actions: When we open a file in read only mode the input pointer is automatically set to beginning so that we can read the file from start. When we open a file in write only mode the existing contents are deleted & output pointer is set to beginning this enables us to write a file from the start. In case we want to open an existing file to add more data, the file is opened in append mode. This moves the output pointer to the end of the file.

Functions for Manipulation of File Pointers:- In the above we discussed the default locations of the pointers for opening file for reading or writing. How do we than move a file pointer to any other desired position inside the file. This is possible only if we can take control of the movement of the file pointer ourselves.

seekg(): Moves get pointer to a specified location.
seekp(): Moves put pointer to a specified location.
tellg(): Gives current position of get pointer
tellp(): Gives current position of put pointer.

DATE 04 10 2024

p→ //Program to Add, Modify, Delete Account class objects & also display Account class object from binary file "acc.dat".

```
#include <stdio.h>
#include <iostream.h>
#include <stdlib.h>
#include <fstream>
using namespace std;
class Account
{
   int ano;
   char anm[15];
   int abal;
   int astate;  // -1-valid / 0-deleted
  public:
   Account()
   {
      ano = abal = 0;
      astate = 1;
      anm[0] = '\0';
   }
   ~Account(){}
   int getNo(){return ano;}
   int getBal(){return abal;}
   int getState(){return astate;}
   char * getName(){return anm;}
   void delRecord(){astate = 0;}
void display()
{
   cout << "\n Account Data : \n No: " << ano << "\n Name: " << anm <<
        "\n Balance: " << abal << "\n State: " << astate;
}
void setData(int);
void setBal(int amt)
{
```

```
abal = abal+ amt;
}
};
void Account :: setData (int no)
{
   ano=no;
   cout << " A/c Name:";
   cin >> anm;
   do
   {
      cout <<"\n Enter Balance :";
      cin >> abal ;
   } while (abal< 500);
   astate= 1;
}

class AccAMD
{
   fstream fa;
   Account ac;
public:
   AccAMD();
  ~AccAMD()
   {
      fa.close();
   }
   int search (int);
   void add();
   void mod();
   void del();
   void display();
   void menu();
};
AccAMD:: AccAMD()
```

DATE 10102024

```
{
  ifstream ain;
  ain.open("acc.dat", ios::in | ios::binary);
  if(!ain)
  {
     ain.close();
     ofstream aout;
     aout.open("acc.dat", ios::out | ios::binary);
     aout.close();
  }
  else
     ain.close();
  fa.open("acc.dat", ios::in|ios::out|ios::binary);
}
int AccAMD :: search(int no)
{
   int pos=0;
   fa.seekg(0, ios::beg); //shifts get ptr to begin of file.
   while(1)
   {
      fa.read((char*)&ac, sizeof(ac));
      if(fa.eof())
      {
         fa.clear(); //unlocks the locked file
         pos=-1;
         break;
      }
      if(ac.getNo()==no)
         break;
      pos=fa.tellg(); //tells current get ptr position from
                      Begin of file.
   }
   return pos;
}
void AccAMD::add()
```

```
{
   int no, pos;
   cout <<"\n A/c No:";
   cin>>no;
   pos= search(no);
   if (pos >=0)
    {
        cout <<"\n Record Exists");
        return;
    }
    ac.setData(no);
    fa.seekp(0, ios::end); //shifts put pointer at byte 0 from End
    fa.write ((char*) &ac, sizeof(ac)); of file i.e. to the End of file.
    cout <<"\n Record Added";
}
// function to Modify Existing Record
void AccAMD::mod()
{
   int no, pos;
   cout <<"\n A/c No:";
   cin>>no;
    pos= search(no);
    if(pos==-1 || ac.getState()==0)
    {
        cout << "\n Record Not Exist");
        return;
    }
    ac.setData(no);
    fa.seekp(pos, ios::beg); //shifts put pointer at byte pos from
    fa.write ((char*) &ac, sizeof(ac));                    begin
    cout <<"Record Modified";
}
void AccAMD :: del()
{
```

```
int no, pos;
cout << "\n A/c No:";
cin >> no;
pos = search(no);
if(pos==-1 || ac.getState()==0)
{
    cout << "\n Record Not Exists";
    return;
}
ac.delRecord();
fa.seekp(pos, ios::beg);
fa.write((char*)&ac, sizeof(ac));
cout << "Record Deleted";
}
void AccAMD :: display()
{
   fa.seekg(0, ios::beg); //shifts get pointer to byte 0 from begin
   while(1)
    {
       fa.read((char*)&ac, sizeof(ac));
       if(fa.eof())
       {
          fa.clear();
          break;
       }
       ac.display();
    }
}
voidAccAMD :: menu()
{
   int opt, pos, no;
   while(1)
   {
      cout << "\n A/C Details \n Menu \n 1: Add New \n 2. Modify \n 3. Delete
```


classmate

```
\n4.Display \n 5. Search \n 6. Exit \n option:";
 cin >>opt ;
if (opt > 5)
   break;
switch (opt)
{
    case 1:
       add(); break;
    case 2:
       mod(); break;
    case 3:
       del(); break;
    case 4:
       display(); break;
    case 5:
       cout << "\n A/C No. to search:";
       cin >> no;
       pos = search(no);
       if (pos == -1)
          cout ("\n Record Not found");
       else
          ac.display();
   }
  }
}

int main()
{
   AccAMD blk;
   blk.menu();
}
```

Difference between text file & binary file.

1) How numbers are stored:

DATE 14/10/2024

- In text file to store 1234 number requires four bytes.
- In binary file to store 1234 no. requires two bytes .i.e. according to datatype.

2] How new line character is managed:
- In text file while writing \n → \n\r while reading \n\r → \n.
- In binary file there is no change.

3] How to end of file is managed:
- The text file ASCII 23 character is used for end of file.
- In binary file the last byte no. is stored in files header part.

4] Access:
- Text File - Sequential Access
- Binary File - Random Access

• Template Functions:-

P→
```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
template <class T>
1. void display (T &var)
{
    cout << "\nData:" << var;
}
template <class Q>
2. void input (Q &var)
{
    cout << "\nData:"; cin >> var;
}
void main()
```

DATE

```
{
   int x;
   input (x);
   display(x);
   double y;
   input(y);
    display(y);
}
```

P → 1.

2.

```
class stu
{
   int no;
   char nm[15];
   public:
      stu() { no=0; nm[0]='\0'; }
     ~stu() { }
      friend istream &operator >> (istream &in, stu &s)
      {
         cout << "\n Roll No.: ";
          in >> s.no;
         cout << "\n Name:";
          in >> s.nm;
          return in;
      }
      friend ostream & operator << (ostream &out, stu &s)
      {
         out <<"\n Roll No:" << s.no << "\n Name:" << s.nm;
         return out;
      }
};
void main()
{
```

DATE

```
int x;
stu y;
input(x); display(x);
input(y); display(y);
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
1.
2.
template <classp>
void tswap (p& a, p& b)
{
    p tmp;
    tmp = a;
     a = b;
     b = tmp;
} (will not work for array's & strings).
void main()
{
   int a, b;
   input (a); input(b);
   display (a); display(b);
   tswap(a,b);
   display (a); display(b);
    double c, d;
    input (c); input (d);
    display (c); display(d);
    tswap (c, d);
    display (c); display(d);
    stu x, y; // Note "stu" class should be added.
    input(x); input(y);
```

PAGE

DATE 15 10 2024

```
    display(x); display(y);
    tswap(x,y);
    display(x); display(y);
}.
```

P→

```
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std; //Overloading of template function.
1.
2.
template <class P>
void tswap (P &a, P&b)
{
    P tmp;
    tmp = a;
     a = b;
     b = tmp;
}
void tswap (char *a , char *b)
{
   char tmp[100];
    strcpy (tmp, a);
    strcpy(a, b);
    strcpy(b, tmp);
}
Void main()
{
   int a = 10, b = 20;
   cout <<"\n A:" <<a <<"\n B:" <<b;
   tswap(a, b);
   cout <<"\n A:" <<a <<"\n B:" <<b;
   char p[] = "Ajinkya";
   char q[] = "Amita";
```

```
    cout << p << "\n" << q << "\n";
    tswap(p,q);
    cout << p << "\n" << q << "\n";
}

p→  #include <stdio.h>
    #include <stdlib.h>
    #include <iostream>
    using namespace std;
    template <class P>
    void input (P*a , int n)
    {
        cout << "\n Data:\n";
        for (int i=0 ; i<n ; i++)
            cin >> a[i];
    }
    template <class T>
    void display (T*a , int n).
    {
        cout << "\nData\n";
        for (int i=0 ; i<n; i++)
            cout << "   " << a[i];
    }
    void main()
    {
        int x[5];
        input (x,5);
        display (x,5);
        double y[6];
        input (y,6);
        display (y,6);
    }
```

DATE 1 8 1 0 2 0 2 4

- Template Classes :-

P →

```
#include <stdio.h>
#include <stdlib.h>
# include <iostream>
using namespace std;
template <class T>
class tclass
{
    T var;
  public:
     void setData()
     {
        cout << "In Data:";
        cin >> var;
     }
     void display()
     {
        cout << "In Data:" << var;
     }
};
void main()
{
   tclass <int> a;
   a.setData();
   a.display();
   tclass <double> b;
   b.setData();
   b.display();
}
```

P →

```
#Include <stdio.h>
#include <stdlib.h>
# include <iostream>
```

DATE

```
using namespace std;
template <class T>
class tclass
{
   T var[5];
  public:
    void setData();
    void display();
};
template <class T>
void tclass <T> ::setData()
{
   cout<<"In Array Data In";
   for(int i=0; i<5 ; i++)
      cin >>var[i];
}
template <class T>
void tclass <T> :: display()
{
   cout <<"In Data In";
   for (int i=0; i<5; i++)
      cout <<"  " <<var[i];
}
void main()
{
   tclass <int> a;
   a.setData();
   a.display();
   tclass <double>b;
   b.setData();
   b.display();
}
```

DATE

P→

```
// Implementation of Stack.
#include <stdio.h>
#include <stdlib.h>
#include <iostream>
using namespace std;
template <class T>
class stack
{
    T var[5]; int top;
  public:
     stack()
     {
        top=-1;
     }
    ~stack() { }
     void push();
     void pop();
     void display();
     void menu();
};
template <class T>
void stack <T> :: push()
{
  if (top==4)
     cout <<"\n Overflow";
  else
   {
     cout <<"\n Data:";
     T val;
     cin >>val;
     top++;
     var[top]= val;
   }
}
```

```
template <class T>
void stack <T> :: pop()
{
  if (top==-1)
      cout<< "\n Underflow";
  else
      top--;
}
template <class T>
void stack<T> :: display()
{
  if (top==-1)
      cout<<"\n Empty stack:";
  else
   {
      cout <<"\n Data:\n";
      for (int i=top; i>=0; i--)
          cout <<" "<< var[i];
   }
}
template <class T>
void stack<T> :: menu()
{
  int opt;
   while(1)
   {
      cout <<"\n Stack Menu\n 1. Push \n 2. Pop \n 3. Display \n 4.
            Exit \n Option:";
      cin >> opt;
      if (opt>3)
         break;
      switch(opt)
      {
         case 1:
```

```
            push(); break;
        case 2:
            pop(); break;
        case 3:
            display();
    }
}
}
```

THE END